

Industry Training

STEGNAR: STEGANOGRAPHIC HUNTER FOR APTS

Submitted for partial fulfilment of the requirements for the award of the degree of

BACHELOR OF ENGINEERING

in

COMPUTER SCIENCE AND ENGINEERING (IoT-CS-BCT)

By

Shiva Naga Chakri M. (2451-24-749-031)
Ayesha Siddiqua Qureshi (2451-24-749-008)
Arjun Thandu (2451-24-749-058)

Under the guidance of

Dr. Sirisha Daggubati
AHoD, CIC (IoT, Cybersecurity, BlockChain)

Department of CSE-ALLIED



MVSR ENGINEERING COLLEGE

(An Autonomous Institution)

Department of Computer Science and Engineering
(Affiliated to Osmania University & Recognized by AICTE)
Nadergul, Balapur Mandal, Hyderabad – 501 510
Academic Year: 2025-2026

MVSR ENGINEERING COLLEGE

(An Autonomous Institution)

Department of Computer Science and Engineering
(Affiliated to Osmania University & Recognized by AICTE)
Nadergul, Balapur Mandal, Hyderabad – 501 510



CERTIFICATE

*This is to certify that the Industry Training work entitled “**STEGNAR: STEGANO-GRAPHIC HUNTER FOR APTS**” is a bonafide work carried out by **Shiva Naga Chakri M. (2451-24-749-031)**, **Ayesha Siddiqua Qureshi (2451-24-749-008)**, **Arjun Thandu (2451-24-749-058)** in partial fulfilment of the requirements for the award of degree of **Bachelor of Engineering in Computer Science and Engineering (IoT-CS-BcT)** from **MVSR Engineering College**, affiliated to **OSMANIA UNIVERSITY**, Hyderabad, during the Academic Year 2025-2026 under our guidance and supervision.*

The results embodied in this report have not been submitted to any other university or institute for the award of any degree or diploma.

Internal Guide

Dr. Sirisha Daggubati
AHoD, CIC (IoT, Cybersecurity &
BlockChain) – Dept. of CSE-Allied

Head of the Department

Dr. Rajesh Kulkarni
Professor & Head
Department of CSE-Allied

DECLARATION

This is to certify that the work reported in the present Industry Training entitled "XXTITLEXX" is a record of bonafide work done by us in the Department of Computer Science and Engineering Allied, M.V.S.R. Engineering College, Osmania University. The reports are based on the work done entirely by us and not copied from any other source.

The results embodied in this report have not been submitted to any other University or Institute for the award of any degree or diploma to the best of our knowledge and belief.

Shiva Naga Chakri M. (2451-24-749-031)

Ayesha Siddiqua Qureshi (2451-24-749-008)

Arjun Thandu (2451-24-749-058)

ACKNOWLEDGEMENT

We would like to express our sincere gratitude and indebtedness to our project guide **Dr. Sirisha Daggubati, AHoD, CIC (IoT, Cybersecurity, BlockChain)** for her valuable suggestions and interest throughout the course of this project.

We are also thankful to our principal and **Dr. Rajesh Kulkarni**, Professor and Head, Department of Computer Science and Engineering-Allied, MVSR Engineering College, Hyderabad for providing excellent infrastructure and a nice atmosphere for completing this project successfully as a part of our B.E. Degree CSE (IoT, Cybersecurity, BlockChain Tech).

We convey our heartfelt thanks to the lab staff for allowing us to use the required equipment whenever needed.

Finally, we would like to take this opportunity to thank our family for their support through the work. We sincerely acknowledge and thank all those who gave directly or indirectly their support in completion of this work.

Shiva Naga Chakri M. (2451-24-749-031)
Ayesha Siddiqua Qureshi (2451-24-749-008)
Arjun Thandu (2451-24-749-058)

VISION AND MISSION

VISION

To evolve as a premier academic and research hub aligned with the global standards in the emerging areas of Computer Science.

MISSION

M1: To impart strong fundamental and applied knowledge, producing technically competent engineers to address current industrial and societal challenges.

M2: To enable students to develop feasible solutions with an awareness towards environment and sustainability.

M3: To nurture a culture of research and innovation that contributes to cutting-edge advancements.

M4: To foster holistic professional development with strong human and ethical values.

ABSTRACT

This project aims to...

Shiva Naga Chakri M. (2451-24-749-031)

Ayesha Siddiqua Qureshi (2451-24-749-008)

Arjun Thandu (2451-24-749-058)

TABLE OF CONTENTS

CERTIFICATE	i
DECLARATION	ii
ACKNOWLEDGEMENT	iii
VISION AND MISSION	iv
ABSTRACT	v

CONTENTS	PAGE NO.S
1 INTRODUCTION & BACKGROUND ANALYSIS	1
1.1 THE EVOLUTION OF ADVANCED PERSISTENT THREATS (APTs)	1
1.1.1 ANATOMY OF AN APT LIFECYCLE	1
1.1.2 THE RISE OF INSIDER THREATS AND COVERT EXFILTRATION	2
1.1.3 CASE STUDY: APT-29 (2015) – Cozy Bear / NOBELIUM . . .	3
1.1.4 FURTHER CASES INVOLVING STEGANOGRAPHIC EXFILTRATION	3
1.2 LIMITATIONS OF CURRENT NETWORK SECURITY PARADIGMS	4
1.2.1 THE OVER-RELIANCE ON CRYPTOGRAPHIC DEFENSES AGAINST OUTSIDE THREATS	4
1.2.2 THE “BLIND SPOT”: WHY TRADITIONAL IDS/IPS FAIL AT COVERT CHANNELS	5
1.3 IDENTIFYING THE MARKET AND SECURITY GAP	6
1.3.1 THE COST OF INVISIBLE EXFILTRATION	6
1.3.2 THE ABSENCE OF OPEN-SOURCE STEGANOGRAPHY DETECTION	7
2 PROBLEM FORMALIZATION & VIABILITY	9
2.1 SCOPE AND OBJECTIVES	9

2.1.1	PRIMARY OBJECTIVES	9
2.1.2	SCOPE BOUNDARIES	10
2.2	CONSTRAINTS AND ASSUMPTIONS	11
2.2.1	COMPUTATIONAL AND LATENCY CONSTRAINTS	11
2.2.2	ATTACKER PROFILE AND MODEL ASSUMPTIONS	11
2.3	SUCCESS METRICS AND FEASIBILITY	12
2.3.1	QUANTITATIVE SUCCESS METRICS	12
2.3.2	TARGET AUDIENCE	12
2.4	FEASIBILITY AND VIABILITY ANALYSIS	12
2.4.1	TECHNICAL FEASIBILITY	12
2.4.2	OPERATIONAL FEASIBILITY	13
2.4.3	MARKET AND ECONOMIC IMPACT	13
3	LITERATURE REVIEW & THEORETICAL FOUNDATIONS	14
3.1	NETWORK STEGANOGRAPHY: MECHANISMS AND VECTORS .	14
3.1.1	PROTOCOL MANIPULATION AND COVERT CHANNELS .	14
3.1.2	PAYLOAD MODIFICATION IN DIGITAL IMAGES	15
3.2	EXISTING DETECTION METHODOLOGY	15
3.2.1	STATISTICAL AND SIGNATURE-BASED STEGANALYSIS	15
3.2.2	MACHINE LEARNING AND CONVOLUTIONAL NEURAL NETWORKS	16
3.3	CRITICAL GAP ANALYSIS	16
4	SOLUTION PROPOSITION & METHODOLOGY	18
4.1	CORE IDEOLOGY OF STEGNAR	18
4.2	GENERAL SOLUTION APPROACH	18
4.2.1	STAGE 1: TRAFFIC CAPTURE	18
4.2.2	STAGE 2: IMAGE CARVING AND VALIDATION	19
4.2.3	STAGE 3: ROUTING AND QUEUE MANAGEMENT	19
4.2.4	STAGE 4: CALPA-NET INFERENCE	19
4.2.5	STAGE 5: SOC DASHBOARD AND ALERTING	20
4.3	THREAT MODELING AND RISK ASSESSMENT	20
4.3.1	ATTACKER PROFILE	20
4.3.2	STRIDE THREAT ANALYSIS	20
4.4	SUMMARY	21
5	SYSTEM ARCHITECTURE	22
5.1	HIGH-LEVEL ARCHITECTURE OVERVIEW	22
5.2	COMPONENT-LEVEL BREAKDOWN	22
5.2.1	Packet Ingestion and Pre-processing Engine	22

5.2.2	Feature Extraction Module: The Routing System (routing-system/)	25
5.2.3	The Steganographic Detection Core	27
5.2.4	Alerting and Logging Interface	30
5.3	RELATIONSHIP AND DATA FLOW ANALYSIS	31
5.3.1	State Transitions and Packet Lifecycle	31
5.4	COMPLEXITY ANALYSIS	33
5.4.1	Computational Complexity	33
5.4.2	Scalability under High Bandwidth	34
5.5	COMPREHENSIVE FEATURE LIST	34
6	IMPLEMENTATION & EXPERIMENTAL SETUP	37
6.1	DEVELOPMENT ENVIRONMENT & TOOLING	37
6.1.1	Infrastructure Stack	37
6.1.2	Test Orchestration Architecture	37
6.1.3	Metrics Collection Pipeline	38
6.2	DATASET GENERATION AND CURATION	39
6.2.1	Base Image Dataset: BOWS2	39
6.2.2	Steganographic Embedding: J-UNIWARD at 0.4 bpp	39
6.2.3	Ground Truth Construction and Traffic Simulation	40
6.3	ALGORITHM IMPLEMENTATION DETAILS	40
6.3.1	CALPA-NET: Channel-Pruning-Assisted SRNet	40
6.3.2	The Python Runtime Bridge	41
6.3.3	Threshold Configuration	41
6.3.4	The gRPC Cycle Window and Stream Reassembly	41
7	EVALUATION & RESULTS	43
7.1	EXPERIMENTAL SCENARIOS AND LOAD SCALABILITY	43
7.2	DETECTION EFFICACY	44
7.2.1	CPU-Constrained Detection Efficacy (Observed)	44
7.2.2	GPU-Enabled Efficacy	45
7.3	SYSTEM PERFORMANCE AND LATENCY PROFILES	46
7.4	HARDWARE RESOURCE UTILIZATION AND SYSTEM OVER-HEAD	47
7.4.1	Endpoint Agent System Resource Profiling	48
7.5	CONTAINERS AND ARCHITECTURAL METRICS	50
7.5.1	Routing and Interception Dynamics	50
7.6	DATABASE AND STORAGE TELEMETRY	51
7.6.1	Queue Saturation Dynamics	52

7.6.2	Forensic Reassembly Overhead (The Tshark Delay)	52
7.7	PROXY INTERCEPTION AND CLIENT DEGRADATION	52
7.8	SYSTEM PERFORMANCE CHARTS	53
7.9	COMPARATIVE BENCHMARKING	56
7.9.1	Stegnar vs Rule-Based IDS (Snort / Suricata)	56
7.9.2	Pruned CALPA-NET vs Full SRNet Baseline	56
7.9.3	Summary	57
8	CONCLUSION & FUTURE DIRECTIONS	58
8.1	SUMMARY OF CONTRIBUTIONS	58
8.2	LIMITATIONS OF THE CURRENT SYSTEM	59
8.3	FUTURE WORK	59
	REFERENCES	61

LIST OF TABLES

4.1	STRIDE Threat Analysis and Mitigation Matrix	21
5.1	Communication protocols used across the system	24
5.2	CALPA-NET Layer Pruning and Keeping Ratios	29
5.3	Detailed Feature List of the Stegnar Platform	34
7.1	CPU-Constrained Detection Efficacy Scaling	44
7.2	GPU-Enabled Efficacy Scaling	45
7.3	System Latency and Throughput Scaling across Node Loads	47
7.4	Server-Side Hardware Resource Utilization and Cache Miss Rates	47
7.5	Endpoint Agent System Resource Profiling	49
7.6	Docker Core Container Resource Metrics (at 100 Nodes Load)	50
7.7	Interception, Routing, and Cache Performance	51
7.8	Database and Storage Telemetry	51
7.9	Stegnar Proxy Interception Network Overhead	53
7.10	Capability Comparison: Snort/Suricata vs Stegnar	56
7.11	Performance Comparison: Full SRNet vs CALPA-NET	57

LIST OF FIGURES

5.1	System Core Ideology and Component Connections	23
5.2	Stegnar Detailed System Architecture Diagram (Multi-Tiered Container Orchestration)	23
5.3	Data Flow Diagram and Process Flowchart for Packet Analysis	32
7.1	Detection Efficacy Metric Comparison (CPU-Constrained vs. GPU- Enabled at 100 Nodes)	46
7.2	Server Hardware Resource Utilization vs. Node Load	48
7.3	Endpoint Agent System Resource Profiling vs. Node Load	49
7.4	Docker Container CPU Utilization Comparison (CPU Mode vs. GPU Mode at 100 Nodes)	50
7.5	Database & Storage Latencies vs. Node Load	52
7.6	Stegnar Proxy Interception Network Latency Overhead	53
7.7	Mean Time to Detect (MTTD) vs. Node Load (CPU vs. GPU)	54
7.8	Effective Throughput vs. Node Load (CPU vs. GPU)	54
7.9	Redis Queue Depth vs. Node Load (CPU vs. GPU)	55
7.10	End-to-End Latency Budget Flowchart	55

Chapter 1

INTRODUCTION & BACKGROUND ANALYSIS

1.1 THE EVOLUTION OF ADVANCED PERSISTENT THREATS (APTS)

The landscape of network security has undergone a fundamental paradigm shift over the last two decades. Early cyber threats were largely characterized by volumetric attacks, opportunistic malware, and noisy intrusions designed for immediate financial gain or system disruption. Today, the most critical risks to enterprise and state infrastructure stem from Advanced Persistent Threats (APTs). Unlike conventional malware, APTs represent highly resourced, often state-sponsored adversaries whose primary objective is prolonged, undetected residency within a target network.

This evolution is defined by the transition from rapid exploitation to low-and-slow asymmetric warfare. Modern APTs prioritize stealth over speed, deliberately throttling their activities to blend with baseline network telemetry. By operating below the detection thresholds of conventional heuristic and signature-based Intrusion Detection Systems (IDS), APTs routinely achieve network dwell times—the duration an attacker remains undetected—ranging from several months to over a year. Consequently, the defense paradigm must shift from merely preventing initial access to proactively hunting for imperceptible anomalous behaviors within internal traffic flows.

1.1.1 ANATOMY OF AN APT LIFECYCLE

To effectively engineer countermeasures against APTs, it is critical to deconstruct their operational methodology. While the specific Tactics, Techniques, and Procedures (TTPs) vary by actor, the overarching lifecycle typically adheres to a structured kill chain:

1. **Reconnaissance and Weaponization:** The adversary conducts passive and active footprinting of the target's external attack surface and open-source intelligence (OSINT). Payloads are custom-compiled to evade known antivirus signatures specific to the target's environment.

-
2. **Initial Compromise:** Execution of the intrusion, most commonly achieved via highly targeted spear-phishing, exploitation of zero-day vulnerabilities in edge-facing infrastructure, or compromised third-party supply chains.
 3. **Establishing a Foothold and C2:** Upon successful execution, the payload establishes a covert Command and Control (C2) channel back to the adversary. This beaconing is often obfuscated via domain fronting or encrypted protocols (e.g., DNS over HTTPS) to mimic legitimate outbound traffic.
 4. **Lateral Movement and Privilege Escalation:** The attacker maps the internal network architecture, moving from the initial point of compromise to higher-value assets. This phase relies heavily on credential dumping (e.g., Mimikatz), Pass-the-Hash techniques, and exploiting Active Directory misconfigurations.
 5. **Data Exfiltration:** The terminal objective. High-value intellectual property, credentials, or state secrets are aggregated, compressed, encrypted, and smuggled out of the network. Because massive outbound data spikes trigger Data Loss Prevention (DLP) alerts, exfiltration is executed incrementally using covert channels.

1.1.2 THE RISE OF INSIDER THREATS AND COVERT EXFILTRATION

As perimeter defenses—such as Next-Generation Firewalls (NGFW) and zero-trust architectures—have matured, external network penetration has become increasingly resource-intensive. In response, threat actors frequently pivot to exploiting insider threats, utilizing either maliciously coerced employees or compromised internal credentials. When the adversary already possesses legitimate authorization within the network boundary, the initial stages of the APT kill chain are entirely bypassed.

In these scenarios, the primary challenge for the adversary is not accessing the data, but exfiltrating it without triggering network telemetry alarms. Traditional DLP solutions rely heavily on deep packet inspection (DPI) looking for known file headers, regex patterns (e.g., credit card numbers), or anomalous volume spikes.

To circumvent this, sophisticated actors have adopted covert exfiltration channels, specifically network steganography. By embedding encrypted payloads within the noise of benign, high-volume traffic—such as altering the least significant bits (LSB) of image payloads or manipulating protocol headers—the exfiltration process becomes mathematically indistinguishable from normal web browsing or media sharing. This renders conventional cryptographic and volumetric defenses completely blind, creating a critical vulnerability in modern network security postures.

1.1.3 CASE STUDY: APT-29 (2015) – Cozy Bear / NOBELIUM

A definitive example of this evolutionary leap in stealth is APT29 (also known as Cozy Bear or NOBELIUM), a threat group attributed to the Russian Foreign Intelligence Service (SVR). APT29 is notorious for its exceptional operational security and pioneering use of obfuscation to bypass enterprise-grade security appliances.

During several high-profile campaigns, including the fallout surrounding the SolarWinds supply chain compromise, APT29 demonstrated a mastery of covert C2 communication. Instead of utilizing traditional beaconing infrastructure, the group leveraged steganography to conceal malicious payloads and configuration files within standard image formats (PNG and JPEG).

In practice, when a compromised internal host requested a seemingly benign image hosted on a legitimate, trusted platform (such as a compromised WordPress site or a social media API), the image downloaded successfully. However, the image contained encrypted C2 instructions embedded within its pixel data. Because the traffic was initiated from inside the network, communicated over standard HTTPS (Port 443), and the payload was a structurally valid image file, traditional IDS engines and web proxies categorized the traffic as safe. This use of steganography allowed APT29 to maintain persistent, invisible access to high-security networks for extended periods.

1.1.4 FURTHER CASES INVOLVING STEGANOGRAPHIC EXFILTRATION

The weaponization of steganography is no longer an isolated phenomenon utilized only by APT29; it has rapidly proliferated across various threat groups as a standard evasion technique:

- **OceanLotus (APT32):** Associated with Vietnamese state interests, this group frequently targeted corporate intellectual property. APT32 utilized custom drop-pers that downloaded standard image files from the internet, extracting highly obfuscated malicious shellcode hidden within the image properties to execute fileless malware directly in memory, leaving zero trace on the physical disk.
 - **The Vawtrak and Zeus Banking Trojans:** Originally designed for financial fraud, variants of these trojans embedded their updated configuration files and stolen financial data within favicons and standard web images. This allowed the malware to continuously update its target lists and exfiltrate credentials while appearing as standard web-caching traffic to network defenders.
 - **Industrial Espionage via "Image Smuggling":** In modern corporate espionage, insiders have been documented utilizing image steganography to exfiltrate propri-
-

etary source code and CAD designs. By embedding compressed intellectual property into high-resolution marketing assets or PDF diagrams and emailing them to external accounts, the insiders bypassed standard corporate DLP scanners, which only validated the structural integrity of the image file without analyzing the entropy of the pixel data.

These cases highlight a systemic failure in current network defense paradigms: the inability to detect structural manipulation within seemingly benign payloads at line speed, underscoring the urgent requirement for deep-packet steganographic introspection.

1.2 LIMITATIONS OF CURRENT NETWORK SECURITY PARADIGMS

1.2.1 THE OVER-RELIANCE ON CRYPTOGRAPHIC DEFENSES AGAINST OUTSIDE THREATS

At the core of modern network security is the assumption that encryption equals safety. We deploy robust protocols like TLS 1.3, IPsec, and end-to-end encrypted tunnels to guarantee confidentiality and data integrity in transit. This setup is highly effective against external adversaries, effectively shutting down Man-in-the-Middle (MitM) attacks, passive sniffing, and unauthorized tampering.

However, this reliance on encryption introduces a critical vulnerability known as the “encryption paradox.” While encryption successfully shields data from external threats, it simultaneously blinds the organization’s own internal security monitoring tools. When an Advanced Persistent Threat (APT) or a malicious insider establishes a foothold inside the network, they inevitably route their exfiltration traffic through standard, encrypted outbound ports — most commonly HTTPS on Port 443.

Because this traffic originates from an authenticated internal host and completes a verified cryptographic handshake, perimeter firewalls assume the connection is benign. Inspecting this traffic requires highly resource-intensive SSL/TLS decryption architectures, which organizations often bypass due to regulatory privacy compliance or heavy CPU overhead. Consequently, the firewall acts as a secure escort, safely delivering the stolen data to the attacker’s Command and Control (C2) server. This exposes a fundamental limitation of cryptography: it secures the transport channel, but provides zero attestation regarding the intent or safety of the payload itself.

1.2.2 THE “BLIND SPOT”: WHY TRADITIONAL IDS/IPS FAIL AT COVERT CHANNELS

Even when organizations implement Deep Packet Inspection (DPI) or SSL decryption, traditional Intrusion Detection and Prevention Systems (IDS/IPS) — such as Snort, Suricata, or commercial Next-Generation Firewalls (NGFW) — are structurally ill-equipped to detect network steganography. This failure is rooted in how these systems process and classify high-speed traffic at the line rate.

Current IDS/IPS solutions rely on three main methodologies, all of which fall short against covert steganographic channels:

1. **Signature-Based Detection:** This approach compares packet payloads against a database of known malicious hashes, byte sequences, or regular expressions (regex). Because steganographic payloads are dynamically generated, modifying the Least Significant Bits (LSB) of a carrier image to embed data alters the file’s hash completely. As a result, the modified file never triggers a signature match.
2. **Protocol Anomaly Detection:** These engines verify whether network traffic conforms to standard RFC protocols. If an attacker embeds a payload within the pixel matrix of a standard JPEG image, the file structure remains completely compliant with the JPEG specification. The application layer (Layer 7 of the OSI model) header stays intact, and the packet traverses the network as a structurally valid, unremarkable media file.
3. **Volumetric and Statistical Analysis:** Data Loss Prevention (DLP) tools typically monitor for sudden spikes in outbound bandwidth. Steganography easily evades this by utilizing a “low-and-slow” exfiltration strategy — breaking the sensitive data into kilobyte-sized fragments and scattering them across gigabytes of normal daily web traffic.

The Computational Bottleneck

Detecting steganography mathematically requires analyzing the localized randomness of the data within the payload. For instance, calculating the Shannon Entropy of an image matrix to spot anomalous density variations involves evaluating:

$$H(X) = - \sum_{i=1}^n P(x_i) \log_2 P(x_i) \quad (1.1)$$

where $P(x_i)$ is the probability of a specific byte value occurring.

The issue is that traditional IDS/IPS architectures are engineered for high-throughput, low-latency environments. Running complex statistical analyses, Fourier

transforms, or pixel-level entropy calculations on every outgoing media packet at 10 Gbps or 100 Gbps line speeds introduces unacceptable network latency.

Because of this computational bottleneck, security appliances enforce a shallow inspection policy for rich media. Once a payload is verified as a structurally valid image file with no known malware signatures, the IDS ceases further algorithmic analysis and passes the traffic. This operational compromise — sacrificing deep statistical inspection to preserve network throughput — creates the definitive “blind spot” that modern APTs exploit for covert data exfiltration.

1.3 IDENTIFYING THE MARKET AND SECURITY GAP

Understanding the technical limitations of current network defenses is only half the equation. To truly justify the engineering of a system like Stegnar, we have to look at the broader operational landscape: what actually happens to an organization when these technical blind spots are exploited, and why hasn’t the cybersecurity market already provided a standardized, accessible solution? This section bridges the gap between the theoretical threat of steganography and the stark operational realities faced by modern Security Operations Centers (SOCs).

1.3.1 THE COST OF INVISIBLE EXFILTRATION

In cybersecurity, we often categorize threats by how “loud” they are. Ransomware, for example, is incredibly loud — it encrypts systems, halts operations, and immediately demands a payout. Covert exfiltration, by contrast, is entirely silent. It is the digital equivalent of a slow leak in a ship’s hull. Because it bypasses traditional Data Loss Prevention (DLP) tools, the most dangerous aspect of steganographic exfiltration is the extended timeline of the breach. When an Advanced Persistent Threat (APT) or a malicious insider successfully establishes a steganographic channel, the resulting costs multiply across several domains:

- **Loss of Intellectual Property (IP) and Competitive Advantage:** The target of covert exfiltration is rarely easily monetizable data like credit card numbers; instead, it is highly valuable, proprietary IP. For example, if an insider slowly trickles aerospace CAD files or pharmaceutical formulas out of a network embedded within routine marketing image uploads, the company loses years of R&D investment. Often, the breach is only discovered months later when a competitor brings an identical product to market.
 - **Regulatory and Compliance Fallout:** Modern regulations like GDPR, HIPAA,
-

and CCPA impose severe financial penalties for exposing PII or PHI. Critically, these frameworks mandate rapid breach disclosure. Because steganographic exfiltration allows attackers to maintain extreme dwell times, organizations often remain unaware of the leakage until the data surfaces on the dark web. This delayed discovery transforms a standard security incident into a catastrophic compliance failure, resulting in multi-million dollar fines.

- **Erosion of Trust and Reputational Damage:** The inability to detect a data hemorrhage severely damages an organization's brand. When clients and partners learn that an attacker operated inside the network for over a year by seamlessly bypassing a multi-million dollar security stack, restoring market confidence is an uphill, often impossible, battle.

Ultimately, traditional security architectures are built to prevent initial intrusions. But when the perimeter inevitably fails, or the threat originates from a trusted insider, the financial survival of the organization depends on detecting the exfiltration itself. When that exfiltration is invisible, the cost is incalculable.

1.3.2 THE ABSENCE OF OPEN-SOURCE STEGANOGRAPHY DETECTION

If the cost of covert exfiltration is so high, we must ask: why aren't deep-packet steganography hunters deployed in every enterprise network? The answer lies in a massive void within the current cybersecurity market. Currently, a dividing line exists between theoretical academic research and practical, deployable software. This disconnect has created the very gap that Stegnar is engineered to fill.

1. **The Academic-to-Industry Disconnect:** Over the last decade, researchers have developed highly accurate deep learning architectures — such as XuNet, SRNet, and various ensemble classifiers — capable of detecting advanced steganographic algorithms like J-UNIWARD. However, these models almost exclusively exist as academic proofs-of-concept. They are written in Matlab or fragmented Python scripts, designed to run against static datasets on a researcher's machine. They are not engineered to ingest live network traffic, handle asynchronous packet queuing, or integrate with enterprise SIEM platforms.
 2. **The Proprietary Black Box:** On the other side of the spectrum is the commercial vendor market. While some high-end, proprietary Next-Generation Firewalls (NGFWs) claim to have deep-packet inspection capabilities, they are closed-source "black boxes." They are prohibitively expensive for mid-sized organizations, academic institutions, and independent researchers. Furthermore, because
-

their detection logic is closed-source, security engineers cannot independently verify how effectively they detect modern, AI-generated steganography, nor can they tune the underlying models to suit their specific network baselines.

3. **The Open-Source Void:** For standard network intrusion detection, the open-source community provides incredibly robust tools. If a SOC wants to detect known malware signatures or protocol anomalies, they can deploy Snort, Suricata, or Zeek for free. These tools are the backbone of modern network defense, completely democratizing baseline security. Yet, there is no “Snort for steganography.” There is no established, open-source pipeline that a security engineer can simply deploy via Docker to start mathematically analyzing image payloads in transit.

This is the exact market and security gap Stegnar targets. By wrapping state-of-the-art deep learning models (like SRNet and XuNet) within a modern, distributed microservices architecture (utilizing gRPC, MinIO, Redis, and TimescaleDB), Stegnar translates theoretical academic research into a practical, open-source tool. It democratizes deep-packet steganographic introspection, giving network defenders an accessible and sophisticated weapon against one of the most elusive threats in the modern cyber landscape.

Chapter 2

PROBLEM FORMALIZATION & VIABILITY

2.1 SCOPE AND OBJECTIVES

The single most important question when building a network security tool is deciding what specific threat we are trying to solve and what we are willing to leave out. Security tools that try to do everything often end up doing nothing well. Stegnar begins with a tightly bounded, specific threat and an architecture designed entirely around addressing it within realistic operational constraints.

The threat we address is the covert exfiltration of sensitive organizational data through steganographically modified image payloads traversing standard enterprise HTTP/TCP traffic. An insider — or a compromised internal host — encodes data into JPEG or PNG images that are visually indistinguishable from the originals, and then uploads them to common destinations like cloud storage, social media, or CDNs. Because this traffic exhibits no anomalous protocol behavior or unusual volume spikes, conventional firewalls, IDS, and DLP systems remain blind to it.

2.1.1 PRIMARY OBJECTIVES

The core objective of the Stegnar project is to design and implement a distributed, microservices-based detection pipeline capable of identifying steganographic manipulation within standard network traffic. Specifically, the system aims to:

- **Decoupled Asynchronous Detection:** Running deep neural network inference on every image payload inline with live traffic is computationally impossible on commodity hardware. A single steganalysis CNN pass can take tens to hundreds of milliseconds, whereas enterprise traffic carries thousands of images per minute. Stegnar decouples packet capture from inference. Capture agents forward extracted images over low-latency gRPC streams to a dedicated routing subsystem, enabling inference to happen asynchronously in the background without degrading network latency.

-
- **Deep Image Introspection via CALPA-NET:** Move beyond standard structural validation of Layer 7 file headers and mathematically analyze the statistical structure of image payloads. We integrate pruned convolutional neural networks (XuNet and SRNet) within the CALPA-NET module. These networks analyze pixel and DCT coefficient distributions to detect the minute statistical “fingerprints” left by adaptive embedding algorithms like J-UNIWARD and UERD.
 - **Real-Time SOC Visibility:** Surfaced alerts must be actionable. Stegnar features a React-based Security Operations Center (SOC) dashboard that displays live alert feeds, anomaly scores, source-destination metadata, and historical trends, translating complex deep-learning outputs into practical security alerts.
 - **Hardware-Independent Deployment:** To ensure accessibility, every component is containerized with Docker and designed to run on standard Commercial Off-The-Shelf (COTS) server hardware, avoiding proprietary NICs or custom FPGAs.

2.1.2 SCOPE BOUNDARIES

Defining what is out of scope is essential to keeping the project manageable and focused.

In Scope:

- Detection of image steganography in JPEG and PNG payloads transported over HTTP/TCP.
- Out-of-band packet capture using lightweight Python-based endpoint agents.
- Transparent redirection of network traffic at the gateway level using eBPF kernel hooks.
- Decoupled, asynchronous backend routing using MinIO (for payload storage) and PostgreSQL (for metadata persistence).
- Real-time alerting via WebSocket and an analyst-facing React dashboard.

Out of Scope:

- *Inline Packet Blocking:* Stegnar is designed strictly as a detection and visibility overlay. It does not perform active blocking; remediation is left to existing network firewalls or EDR software.
-

-
- *TLS Decryption*: Stegnar does not attempt to break encryption. It assumes it resides behind an enterprise SSL/TLS inspection proxy or that endpoint agents capture payloads before they are encrypted.
 - *Non-Image Media*: VoIP, video streams, audio, and documents (PDF, DOCX) are excluded. Focus is placed entirely on image matrices to optimize CNN detection performance.

2.2 CONSTRAINTS AND ASSUMPTIONS

2.2.1 COMPUTATIONAL AND LATENCY CONSTRAINTS

Steganalysis neural networks are computationally expensive. A deep residual network like SRNet can take 80–200 ms to process a single image. At enterprise scale, inline analysis would cause severe network degradation. Stegnar’s asynchronous pipeline resolves this: carved images are placed in an object storage queue, and the CALPA-NET worker processes them at its own rate. Since exfiltration is historically a “low-and-slow” campaign, a detection delay of a few minutes is an acceptable operational trade-off.

Furthermore, userspace packet capture introduces significant CPU overhead due to kernel-to-userspace context switches. The MITM gateway mitigates this by utilizing eBPF programs to redirect traffic at the kernel level. On the host endpoints, userspace Python capture agents are throttled using rate limiters to prevent CPU starvation.

2.2.2 ATTACKER PROFILE AND MODEL ASSUMPTIONS

To evaluate detection efficacy, we assume the following about the adversary:

- **Embedding Algorithms**: The adversary uses state-of-the-art content-adaptive embedding algorithms, specifically J-UNIWARD, S-UNIWARD, or UERD. These algorithms hide data in high-frequency, complex regions of the image to minimize statistical noise.
 - **Traffic Accessibility**: Outbound traffic is either decrypted at a TLS-terminating firewall before reaching the Stegnar gateway, or captured locally by endpoint agents before encryption.
 - **Adversarial Capacity**: The adversary is assumed to use standard research-grade steganographic tools at low embedding rates (0.1–0.4 bpnzac) to blend with normal traffic. Active probing or evasion of the neural network’s specific weights is out of scope.
-

2.3 SUCCESS METRICS AND FEASIBILITY

2.3.1 QUANTITATIVE SUCCESS METRICS

To evaluate Stegnar’s performance, the following quantitative benchmarks are established:

- **Detection Efficacy:** Achieve a True Positive Rate (Recall) of $\geq 90\%$ at an embedding rate of 0.4 bpnzac, and $\geq 70\%$ at 0.2 bpnzac.
- **False Positive Rate:** Maintain an FPR of $\leq 5\%$ against clean corporate web traffic to minimize alert fatigue.
- **Throughput:** Achieve a pipeline processing rate of ≥ 500 images per minute on a standard single-GPU server, with a 95th-percentile end-to-end latency of under 30 seconds.

2.3.2 TARGET AUDIENCE

The system is designed for three main groups:

1. **SOC Analysts:** Non-expert security personnel who require a simplified, visual interface displaying alerts, metadata, and anomaly scores.
2. **Network Administrators:** Engineers who deploy and maintain the containerized microservices and configure the eBPF redirect filters.
3. **Steganalysis Researchers:** Academics who want to evaluate new models on a live network-integrated testbed rather than static offline datasets.

2.4 FEASIBILITY AND VIABILITY ANALYSIS

2.4.1 TECHNICAL FEASIBILITY

The system utilizes mature, production-grade tools: Docker for microservices, gRPC for streaming, MinIO for storage, and PostgreSQL for metadata. The primary technical challenge is the carving module, which reassembles fragmented TCP packets into valid JPEGs/PNGs by scanning for magic bytes. This is a well-understood parsing problem, making the pipeline highly viable on standard COTS hardware.

2.4.2 OPERATIONAL FEASIBILITY

Stegnar fits seamlessly into standard enterprise networks. The MITM gateway acts as a transparent bridge, requiring no changes to endpoint networking. Endpoint agents run as lightweight background daemons. System alerts are exported via webhooks to standard SIEMs (like Splunk or Elastic), integrating with existing incident response workflows.

2.4.3 MARKET AND ECONOMIC IMPACT

With the average data breach cost exceeding \$4 million (according to the IBM Cost of a Data Breach Report), detecting stealthy insider exfiltration early represents a major cost saving. The current cybersecurity market lacks real-time, network-integrated steganography tools; existing utilities are mostly offline forensics tools. By providing an open-source, Dockerized pipeline, Stegnar democratizes network steganalysis for organizations that cannot afford expensive, proprietary black-box firewalls.

Chapter 3

LITERATURE REVIEW & THEORETICAL FOUNDATIONS

3.1 NETWORK STEGANOGRAPHY: MECHANISMS AND VECTORS

Network steganography — encoding hidden information within network communications such that the traffic appears indistinguishable from normal flows — is a broader category than classical file-based steganography. While classical steganography modifies a static carrier (like a local image or audio file), network steganography exploits the structure of live communication protocols.

3.1.1 PROTOCOL MANIPULATION AND COVERT CHANNELS

Some of the earliest network steganography techniques manipulated unused or optional fields within network headers. Murdoch and Lewis (2005) demonstrated that the IP Identification (IPID) field — a 16-bit value used for IPv4 packet fragmentation reassembly — could be systematically altered to carry hidden data without affecting packet delivery.

Beyond IPID manipulation, Rowland (1997) showed that TCP Initial Sequence Numbers (ISNs) could serve as a storage channel, and Ahsan and Kundur (2002) analyzed the bandwidth capacities of various IP and TCP header fields. Additionally, timing-based covert channels, as surveyed in the taxonomy by Mazurczyk et al. (2016), hide data in inter-packet arrival intervals rather than packet headers. Since no packet payload is modified, timing channels bypass deep packet inspection completely.

Other approaches include the HICCUPS system (Szczypiorski, 2003), which exploits corrupted frames in local area networks, and LACK (Mazurczyk and Kotulski, 2006), which hides data in lost audio packets in VoIP streams. While protocol-level anomalies are out of scope for Stegnar, understanding them is crucial for mapping the broader covert exfiltration landscape.

3.1.2 PAYLOAD MODIFICATION IN DIGITAL IMAGES

Payload-based steganography, which hides information directly within the application data, is the specific target of the Stegnar pipeline.

Spatial Domain Steganography: Naive Least Significant Bit (LSB) substitution overwrites the LSBs of pixel intensity values with message bits. As analyzed by Chan and Cheng (2004), naive LSB substitution introduces sharp statistical anomalies in pixel histograms, making it easily detectable. LSB matching (Mielikainen, 2006) mitigates this by randomly adding or subtracting 1 from pixel values instead of directly overwriting them. More advanced spatial-domain methods, such as Wavelet Obtained Weights (WOW, Holub and Fridrich, 2012), calculate spatial complexity across the image matrix to concentrate changes in noisy, textured regions where statistical distortions are less visible.

Frequency Domain Steganography: Modern high-security steganography operates in the frequency domain, modifying the Discrete Cosine Transform (DCT) coefficients of JPEG images to avoid double-compression artifacts.

J-UNIWARD (JPEG Universal Wavelet Relative Distortion, Holub and Fridrich, 2013) models steganographic embedding as an optimization problem: it minimizes relative distortion in the wavelet domain, concentrating modifications in high-frequency, complex coefficients. Similarly, UERD (Uniform Embedding Revisited Distortion, Guo et al., 2015) improves security by distributing embedding changes more uniformly across DCT coefficients, resisting statistical detectors designed to seek clustered distortions. These adaptive embedding methods rely on Syndrome-Trellis Codes (STCs, Filler et al., 2011) to achieve near-optimal coding efficiency, minimizing the physical changes required to hide a given payload.

3.2 EXISTING DETECTION METHODOLOGY

3.2.1 STATISTICAL AND SIGNATURE-BASED STEGANALYSIS

Early steganalysis relied on signature matching. Westfeld and Pfitzmann (1999) introduced the chi-square attack, which successfully detects sequential LSB embedding in GIF files by identifying the statistical equalization of adjacent histogram pairs. Fridrich and Goljan (2002) proposed calibration-based steganalysis, which estimates the original cover image's DCT statistics by cropping and re-compressing the stego image.

To generalize detection across unknown embedding schemes, researchers developed the Spatial Rich Model (SRM, Fridrich and Kodovský, 2012) and the JPEG Rich Model (J+SRM, Kodovský and Fridrich, 2012). These models extract high-dimensional co-

occurrence matrices (exceeding 34,000 features) from image residuals and train Fisher Linear Discriminant (FLD) ensembles to distinguish cover from stego images. However, rich models are computationally heavy and require manual feature engineering, making them vulnerable to newer adaptive cost functions designed specifically to bypass them.

Furthermore, traditional Rule-Based IDS (like Snort and Suricata) cannot inspect statistical image structures. They are designed to match byte signatures in headers or plaintext payloads, leaving them blind to statistically modified coefficients inside valid media files.

3.2.2 MACHINE LEARNING AND CONVOLUTIONAL NEURAL NETWORKS

Convolutional Neural Networks (CNNs) revolutionized steganalysis by automatically learning discriminative residual features directly from training data.

XuNet: Developed by Xu, Wu, and Zhang (2016), XuNet initialized its first layer with high-pass filters to extract noise residuals, forcing the model to focus on steganographic changes rather than semantic image content.

Ye-Net: Ye, Ni, and Yi (2017) introduced a truncated linear activation function to prevent large image edges from dominating the residual signals, improving spatial detection accuracy.

SRNet: Boroumand et al. (2019) proposed Steganalysis Residual Network (SRNet), which remains the state of the art in JPEG-domain steganalysis. SRNet bypasses manual high-pass pre-filtering, using a deep residual network where the first several layers omit pooling to preserve fine-grained DCT-residual statistics, while subsequent blocks aggregate local representations into binary classifications.

CALPA-NET: A major drawback of SRNet is its large VRAM and compute footprint. CALPA-NET (Chen et al., 2021) resolves this by applying structured channel pruning to the trained SRNet. Channel pruning identifies and removes redundant filters, reducing inference latency and parameter size by $4\text{--}5\times$ without proportional accuracy degradation. Stegnar implements the pruned CALPA-NET architecture as its core inference engine.

3.3 CRITICAL GAP ANALYSIS

A critical review of the literature reveals a significant disconnect between steganalysis theory and practical network engineering:

-
1. **The Offline-to-Online Gap:** Virtually all published steganalysis research (including SRM, SRNet, and CALPA-NET) is benchmarked using static, pre-labeled image datasets (e.g., BOWS2 or BOSS Base). These benchmarks ignore the complexities of live networks, such as TCP packet fragmentation, streaming delays, SSL/TLS decryption requirements, and SIEM alerting interfaces.
 2. **The Open-Source Void:** While open-source IDS like Snort and Zeek democratize basic intrusion detection, there is no open-source, network-integrated tool for steganography detection. Existing utilities are offline, file-system forensic scripts designed for post-incident review rather than live network stream carving and parallel inference.

Stegnar addresses this exact gap by wrapping state-of-the-art CNNs inside a distributed, open-source microservices pipeline, transforming theoretical steganalysis into a practical enterprise defense tool.

Chapter 4

SOLUTION PROPOSITION & METHODOLOGY

4.1 CORE IDEOLOGY OF STEGNAR

Traditional enterprise network security is perimeter-focused: firewalls inspect headers, IDS engines match signatures of known malware, and DLP scanners look for credit card numbers or keywords in text. These methods are blind to the statistical structure of image files. A steganographic payload is wrapped inside a structurally perfect image file, traversing whitelisted protocols (like HTTP/HTTPS) to routine endpoints (like cloud storage or CDNs).

Stegnar shifts the paradigm from perimeter defense to deep semantic payload analysis. By integrating steganalysis CNNs directly into the network traffic path, the system analyzes the statistical distributions of pixels and DCT coefficients. This does not replace existing tools but introduces a complementary layer specifically calibrated for covert exfiltration.

In doing so, Stegnar prioritizes operational pragmatism. It decouples high-overhead neural network execution from the live traffic path, establishing an asynchronous, software-defined pipeline that can run on standard COTS hardware.

4.2 GENERAL SOLUTION APPROACH

Stegnar is structured as a sequential processing pipeline with cleanly separated microservices. The pipeline consists of five stages.

4.2.1 STAGE 1: TRAFFIC CAPTURE

Traffic ingestion operates across two parallel vectors, covering different trust boundaries:

- **Endpoint Agents (PCAP Sniffers):** Python-based background daemons deployed on user endpoints. Using libpcap bindings, agents capture outbound TCP

streams, apply BPF filters to isolate HTTP/TCP payloads, and stream raw image bytes over gRPC to the central coordinator. Endpoint agents capture payloads at the application layer before they are encrypted, bypassing TLS visibility limitations.

- **MITM Gateway (eBPF-Based Transparent Proxy):** A transparent network bridge deployed on the egress path. The gateway uses eBPF kernel hooks (`ebpf_redirect.sh`) to intercept and steer traffic from the kernel's network path directly to a userspace reassembly socket. Redirection at the kernel level minimizes userspace context-switch overhead. The gateway assumes SSL termination occurs at or before this point (via an enterprise TLS proxy).

4.2.2 STAGE 2: IMAGE CARVING AND VALIDATION

Images are rarely transmitted in single packets. The carving module performs TCP stream reassembly, scanning for JPEG file markers (SOI: FFD8FF, EOI: FFD9) and PNG signatures (89 50 4E 47 0D 0A 1A 0A) to carve complete media payloads out of raw byte streams. Validated image payloads are saved in MinIO object storage, keyed by their SHA-256 content hashes, while connection metadata is recorded in PostgreSQL as a pending inference job.

4.2.3 STAGE 3: ROUTING AND QUEUE MANAGEMENT

The routing system manages the ingestion flow to protect the inference engine from queue exhaustion:

- **Priority Scheduling:** Payloads originating from high-risk watchlists are prioritized.
- **Deduplication:** Identical image hashes are processed once. Multiple metadata records are linked to a single cached inference verdict.
- **Load Management:** If the Redis queue depth exceeds a high-water mark, the system drops low-priority payloads, preferring graceful degradation over database crashes.

4.2.4 STAGE 4: CALPA-NET INFERENCE

Pending inference jobs are consumed by the CALPA-NET worker:

-
1. **Preprocessing:** JPEGs are parsed to extract raw DCT coefficients (for SRNet), preserving embedding statistics. For XuNet, images are decoded to normalized spatial tensors.
 2. **CNN Inference:** Pruned SRNet models evaluate DCT tensors (JPEG domain), while XuNet models evaluate spatial pixels.
 3. **Verdicts:** Model probabilities are aggregated via a weighted average into a composite anomaly score $S \in [0, 1]$, which is written back to PostgreSQL.

4.2.5 STAGE 5: SOC DASHBOARD AND ALERTING

The SOC API broadcasts completed evaluations via WebSockets to a React dashboard. Analysts view:

- A real-time alerts feed with anomaly scores, source-destination metadata, and image previews.
- Network topology maps highlighting high-risk host connections.
- Options to review, escalate, or dismiss alerts. Alerts exceeding $S > 0.7$ trigger external SIEM webhooks.

4.3 THREAT MODELING AND RISK ASSESSMENT

4.3.1 ATTACKER PROFILE

The adversary is defined as a malicious insider or compromised host attempting covert data exfiltration. The attacker embeds sensitive data (credentials, IP, PII) into cover images using J-UNIWARD or UERD at low embedding rates (0.1–0.4 bpnzac) to avoid visual and statistical anomalies. Payloads are uploaded infrequently to whitelisted business domains (like Dropbox or Google Drive) to blend with normal corporate traffic, bypassing traditional volumetric firewalls.

4.3.2 STRIDE THREAT ANALYSIS

To secure the Stegnar platform itself, we evaluated the system against the STRIDE threat framework. Table 4.1 outlines the threat categories, specific risks, and architectural countermeasures.

Table 4.1: STRIDE Threat Analysis and Mitigation Matrix

Threat Category	Specific Risk	Stegnar Countermeasure
Spoofing	Adversary spoofing endpoint source IP addresses.	Log network-layer metadata at capture; cross-correlate agent and gateway records.
Tampering	Intermediate proxy re-compression destroying stego artifacts.	Perform extraction at endpoint agent before proxy compression; log modified scores.
Repudiation	Attacker denying data exfiltration activities.	Maintain immutable PostgreSQL audit logs linked to MinIO SHA-256 payload hashes.
Info. Disclosure	Leaking captured image payloads or SOC metadata.	Restrict MinIO bucket access; enforce role-based access control (RBAC) on SOC APIs.
Privilege Elev.	Compromising the API or inference worker containers.	Isolate microservices using Docker; run backend services without host network access.
Denial of Serv.	Flooding the pipeline with traffic to exhaust memory.	Implement sliding-window rate limiters at proxy and drop-policies at Redis queues.

4.4 SUMMARY

By transitioning from perimeter protection to semantic analysis of image payloads, the Stegnar architecture targets a critical gap in contemporary network security. Through eBPF kernel offloading and asynchronous processing queues, it achieves high scalability on standard enterprise hardware. In Chapter 5, we present the structural and programmatic details of this implementation.

Chapter 5

SYSTEM ARCHITECTURE

5.1 HIGH-LEVEL ARCHITECTURE OVERVIEW

Stegnar is built on one foundational conviction: no analysis should ever block the network path. Steganography detection using deep learning is expensive — a single SRNet inference pass, even on GPU, takes tens to hundreds of milliseconds. Attempting this inline would introduce visible latency, tip off adversaries, and collapse under any real enterprise traffic load. The architecture’s answer to this is a fully decoupled, asynchronous microservices pipeline where every component does exactly one thing and passes its output to the next stage without waiting.

The system is composed of five logical tiers: a distributed interception layer that passively captures network traffic across endpoints; a central routing system that re-assembles, deduplicates, and orchestrates analysis jobs; a compute-intensive inference cluster that runs the pruned CALPA-NET steganalysis model; a persistence layer that commits forensic artifacts to immutable storage; and a SOC-facing dashboard that surfaces findings in real time.

All services are containerized via Docker Compose and communicate over a private `stegnar-net` bridge network. The three underlying infrastructure components — Redis 7 (message queue + cache), TimescaleDB/PostgreSQL 15 (time-series forensic ledger), and MinIO (S3-compatible object store) — form the stateful backbone that every stateless service reads from or writes to.

5.2 COMPONENT-LEVEL BREAKDOWN

5.2.1 Packet Ingestion and Pre-processing Engine

Stegnar supports two complementary interception modes, both of which ultimately funnel traffic into the same Routing System gRPC endpoint.

Endpoint Agent (`endpoint-agent/`)

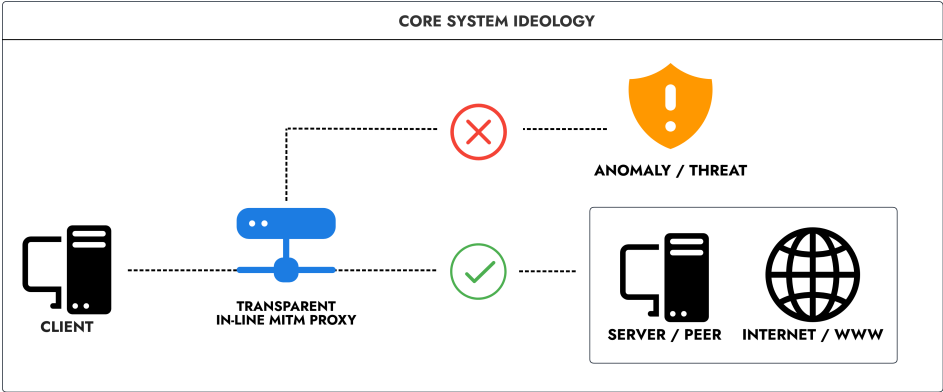


Figure 5.1: System Core Ideology and Component Connections

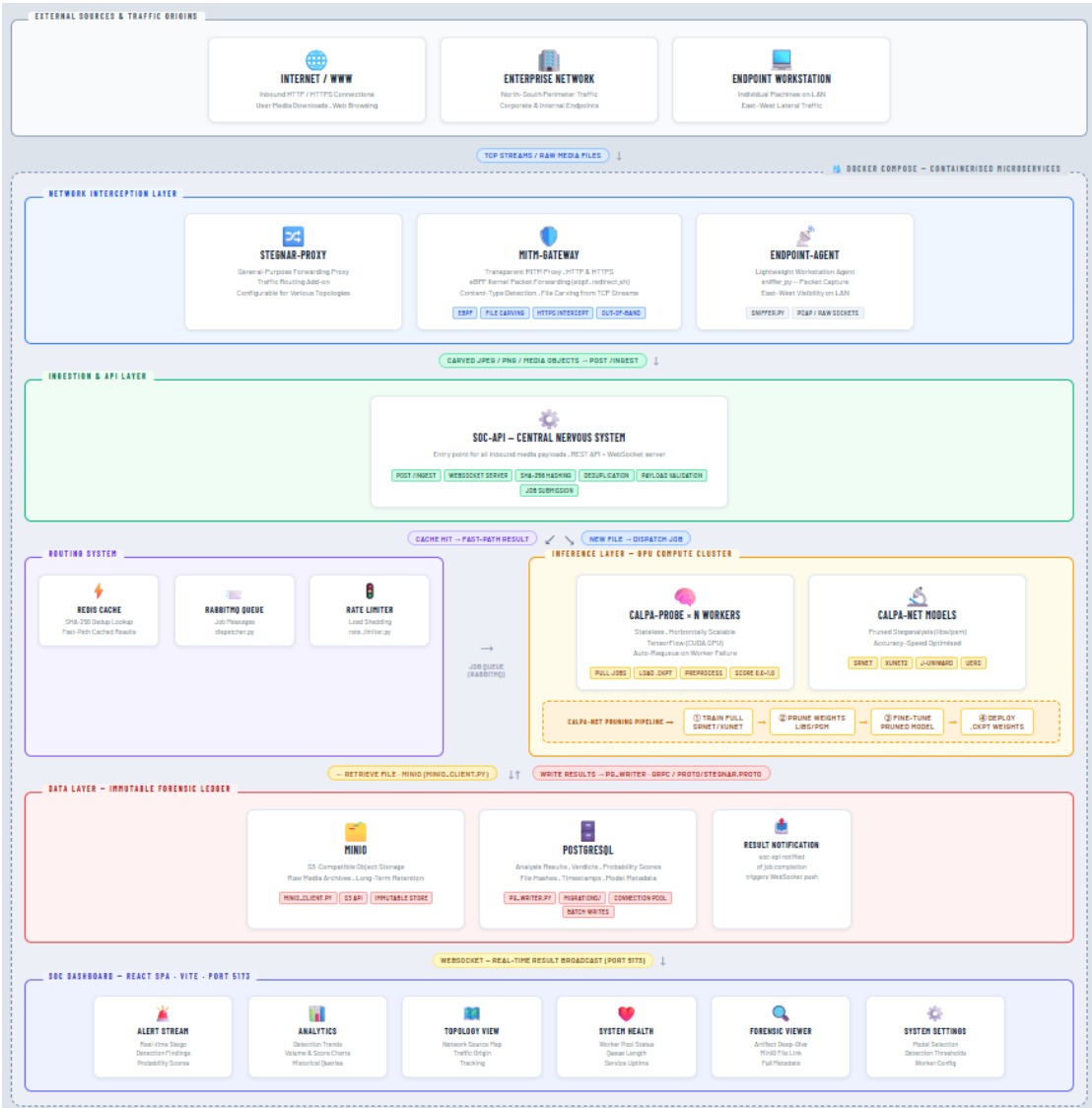


Figure 5.2: Stegnar Detailed System Architecture Diagram (Multi-Tiered Container Orchestration)

Table 5.1: Communication protocols used across the system

Protocol	Used Between	Purpose
gRPC / HTTP/2	Endpoint Agent → Routing System	Streaming raw packet chunks and TLS keys
gRPC / HTTP/2	Routing System → MITM Gateway	Dispatching carved image for inference
Redis Streams (XADD/XREADGROUP)	Routing System → Data Layer	Asynchronous persistence queue
REST / HTTP	SOC API ↔ Frontend, Agent Heartbeats	Queries, config, health checks
WebSocket	SOC API ↔ Frontend Dashboard	Real-time verdict streaming

The endpoint agent is a lightweight Python daemon deployed on monitored workstations — installed as a `systemd` service on Linux or a Scheduled Task on Windows via the scripts in `agent-installer/`. Its design is deliberately minimal: it does no analysis whatsoever. Its only jobs are capturing packets and shipping them upstream.

Internally, the agent runs four concurrent `asyncio` tasks that operate independently:

1. **Packet Sniffer** (`sniffer.py`) — Attaches a Scapy `AsyncSniffer` to the configured network interface (default `eth0`) with a BPF filter that excludes Stegnar’s own internal gRPC ports (50051, 50052, 8080, 9000) to avoid feedback loops. For every IP packet carrying a non-empty TCP or UDP payload, it instantiates a `CapturedPacket` dataclass containing the raw IP-layer bytes, a pre-computed SHA-256 hash, the five-tuple (src IP, dst IP, src port, dst port), and a millisecond-precision capture timestamp. This object is pushed thread-safely into an `asyncio` queue using `loop.call_soon_threadsafe`.
 2. **SSL Key Extractor** (`key_extractor.py`) — Many browsers and curl expose TLS session keys via the `SSLKEYLOGFILE` environment variable, writing NSS-format key material to a log file in real time. The key extractor tail-follows this file with a 100ms polling interval, pushing each new non-comment line into a separate `asyncio` key queue. These keys are later forwarded alongside raw packet bytes so the routing system can decrypt HTTPS traffic for image extraction.
 3. **gRPC Stream Client** (`grpc_client.py`) — Rather than maintaining one permanent gRPC stream (which would prevent the server from processing accumulated data), the client operates in **cyclic 20-second sessions**. Each cycle opens an `insecure_channel` to the routing system, drains both packet and key queues, constructs `PayloadChunk` protobuf messages, and streams them over
-

`RouterService.StreamPayload`. When the 20-second deadline hits, the stream generator simply returns — causing the server’s streaming RPC to complete, which triggers the routing system to flush and process everything received in that window. After a 3-second cooldown, the cycle repeats. This design guarantees regular, predictable processing without holding analysis work in limbo.

4. **Heartbeat Loop** — Every 15 seconds, a simple HTTP POST is fired to the SOC API’s `/api/agents/heartbeat` endpoint with the agent’s endpoint ID and resolved IP address. This keeps the `endpoint_registry` table current even during idle periods when no interesting traffic is flowing.

Transparent Proxy (`stegnar-proxy/`)

For network segments where deploying an agent on every host isn’t feasible, Stegnar provides an alternative: `stegnar-proxy`, which runs a `mitmproxy` instance in transparent mode on port 8080. The `StegnarAddon` class hooks into `mitmproxy`’s `request()` and `response()` callbacks. For each HTTP flow, it checks both the `Content-Type` header and the first bytes of the payload against a table of image magic signatures (JPEG: `\xff\xd8\xff`, PNG: `\x89PNG\r\n`, WebP: `RIFF`, BMP: `BM`, GIF: `GIF87a/GIF89a`). If a match is found and the payload exceeds 1,000 bytes (to avoid thumbnails and icons), the raw image bytes are packaged as a `PayloadChunk` and pushed to an internal `asyncio` queue. A background `stream_worker` coroutine continuously streams these chunks to the routing system over a persistent gRPC channel with automatic reconnection on failure. Notably, because `mitmproxy` has already decrypted the HTTPS traffic on the fly, no TLS key material needs to accompany the payload — the image bytes arrive in plaintext.

5.2.2 Feature Extraction Module: The Routing System (`routing-system/`)

The routing system is the central nervous system of the platform. It listens on gRPC port 50051, orchestrates every incoming packet stream, and decides what to do with each payload. At its core is `RouterServicer`, which implements the `RouterService.StreamPayload` RPC — a client-streaming call where the agent sends chunks and the server returns a single `StreamAck` once the stream closes.

Stream Reassembly and the Reaper

Because TCP connections are inherently fragmented, individual packets carrying different chunks of the same image arrive as separate `PayloadChunk` messages. The servicer groups these by `stream_id` — a direction-normalized string of the form `src_ip:src_port-dst_ip:dst_port` — into an in-memory dictionary called `self`.

`_streams`. Each stream entry accumulates raw byte chunks alongside metadata from the first chunk seen.

A background reaper task runs every 5 seconds. Any stream that hasn't received a new chunk in the past 5 seconds is considered complete, removed from the active map, and passed to `_process_stream()` for full analysis. Additionally, when a gRPC streaming call completes (i.e., the agent's 20-second cycle ends), all streams touched during that call are immediately flushed, regardless of inactivity age. This dual approach ensures both timely processing and coverage of short-lived streams.

Pipeline Stages in `_process_stream()`

Once a stream is flushed, it goes through a structured seven-stage pipeline:

- **Stage 1 — Endpoint Registry Upsert:** The endpoint's record in the PostgreSQL `endpoint_registry` table is updated with the latest IP address and a delta of bytes received, giving SOC operators a live view of all reporting agents.
 - **Stage 2 — Forensic PCAP Construction (`pcap_builder.py`):** This is one of the more sophisticated steps. The raw byte chunks are passed to `PcapBuilder.build_pcap()`, which runs synchronously in a thread pool executor to avoid blocking the event loop. Scapy parses each chunk — checking for the IPv4 header magic byte `0x45` before attempting to parse — and writes valid packets to a temporary `.pcap` file using `wrpcap`. If TLS session keys are available from the KeyStore for this endpoint, they are written to a separate key file. The builder then invokes `tshark` with the `--export-objects http,<dir>` flag (and `--tls.keylog_file` if keys are available) to decrypt the stream and extract HTTP objects. If `tshark` finds nothing (e.g., for non-HTTP traffic or missing keys), a fallback regex scan looks for the JPEG start-of-image marker (`\xff\xd8\xff`) and end-of-image marker (`\xff\xd9`) in the raw byte stream and carves the image manually. Both the `.pcap` file and any extracted image are uploaded to MinIO (`stegnar-pcaps` and `stegnar-artifacts` buckets respectively), and their object URIs are recorded for later reference.
 - **Stage 3 — SHA-256 Hash Deduplication (`cache.py`):** The full SHA-256 of the reassembled payload is computed and checked against a Redis hash cache keyed as `stegnar:cache:<sha256>`. Each cache entry stores the verdict and probability score, with a TTL of one week. On a cache hit, the result is immediately written to the event queue — no ML inference is needed. This is a critical optimization: in real networks, the same images (favicons, logos, assets) appear constantly, and skipping repeat analysis prevents the inference cluster from being flooded with redundant work.
-

-
- **Stage 4 — MIME Sniffing:** If the cache misses, the reassembled payload is inspected for image magic bytes. Either the raw bytes directly start with a known image signature (e.g., from the proxy sending pre-carved data), or the tshark-carved bytes from Stage 2 are used. If no image content is detected at all, the event is logged as `NO_IMAGE` and the pipeline terminates without a CALPA dispatch.
 - **Stage 5 — Rate Limiting** (`rate_limiter.py`): Before dispatching to the inference cluster, each chunk is checked against a per-endpoint sliding-window rate limiter implemented in Redis using a sorted set (key: `stegnar:rl:<endpoint_id>`). The window is 1 second with a default limit of 1,000 chunks. Requests that exceed the limit are silently dropped. This protects the MITM gateway from being overwhelmed by a single malfunctioning or compromised endpoint. The maximum number of concurrent in-flight inference requests is also governed by a Redis-synced `maxConcurrentScans` setting, checked every 10 seconds by a background task. If the concurrency ceiling is reached, the stream processing coroutine simply waits in 500ms polling intervals until a slot opens.
 - **Stage 6 — MITM Dispatch** (`dispatcher.py`): The `MITMDispatcher` wraps a persistent async gRPC channel to `mitm:50052`. It constructs an `AnalysisRequest` protobuf containing the stream ID, SHA-256, source/destination IPs, and the raw image bytes (up to 50 MB, the configured gRPC message size limit), then calls `AnalysisService.AnalyzeImage`. If the gRPC call fails with `AioRpcError`, a synthetic `ERROR` verdict is returned rather than crashing — ensuring the pipeline remains alive even when the inference cluster is temporarily unavailable.
 - **Stage 7 — Result Persistence:** The `AnalysisResult` is stored back into the Redis hash cache for future deduplication, persisted to the `hash_cache` PostgreSQL table, and appended to the `stegnar:db_queue` Redis Stream via `QueueWriter.write_event()`. The Redis Stream entry contains the full flat record: stream ID, endpoint, hash, verdict, confidence score, latency, PCAP URI, image URI, and a millisecond epoch timestamp.

5.2.3 The Steganographic Detection Core

MITM Gateway (`mitm-gateway/`)

The MITM Gateway is a dedicated gRPC server — implemented in async Python 3.11 — that exposes the `AnalysisService` on port 50052. Its sole handler, `AnalysisServicer.AnalyzeImage`, receives an `AnalysisRequest`, calls `analyze_`

`image()` from `calpa_runner.py`, and returns an `AnalysisResult` proto containing the verdict (CLEAN — STEGO — ERROR), confidence score (0.0–1.0), raw logit, model type, and inference latency in milliseconds.

The Python Version Bridge (`calpa_runner.py`)

There's a practical engineering constraint at the heart of the inference cluster: the CALPA-NET model was built on TensorFlow 1.15, which only runs under Python 3.7. The gateway itself, however, runs Python 3.11 (required for modern asyncio and gRPC APIs). Rather than downgrading the entire service, the design uses a **subprocess bridge**.

When `analyze_image(image_bytes)` is called, the image bytes are written to a temporary JPEG file on disk. A JSON payload is assembled:

```
{
  "image_path": "/tmp/tmpXXXXXX.jpg",
  "model_path": "/calpa/generated_cfg_and_model/
                 trained_pruned_model/Model_438375.ckpt",
  "cfg_path":   "/calpa/generated_cfg_and_model/
                 srnet_juniward_04_threshold05.cfg",
  "threshold": 70
}
```

This is piped over stdin to `calpa_worker.py`, which runs under `/opt/tf1/bin/python3.7`. The subprocess is launched asynchronously via `asyncio.create_subprocess_exec`, with a configurable timeout (default 120 seconds). On completion, the worker's stdout is parsed to extract the JSON result. The temporary file is cleaned up in a `finally` block regardless of outcome. The threshold value (default 70, meaning 70% probability of steganography) is read live from `stegnar:settings` in Redis, allowing SOC operators to tune sensitivity from the dashboard without restarting the service.

The Inference Worker: CALPA-NET (`calpa_worker.py`)

This is the most technically substantial file in the project. It implements the complete inference pipeline for the pruned SRNet model targeting J-UNIWARD steganography at 0.4 bits per pixel embedding rate.

Pruning Configuration Parsing (`_read_pruned_channels`): The pruning architecture is defined in `srnet_juniward_04_threshold05.cfg`, a two-section INI file that specifies the fraction of channels *pruned* (removed) for each layer under two complementary algorithms: ThiNet (layers 3–12) and L1-norm (layers 8–12). The worker inverts these to compute the fraction of channels *kept*:

Table 5.2: CALPA-NET Layer Pruning and Keeping Ratios

Layer	ThiNet Prune Rate	Channels Kept (ThiNet)	L1 Prune Rate	Channels Kept (L1)
3	10%	90%	—	—
4	50%	50%	—	—
5	50%	50%	—	—
6	40%	60%	—	—
7	5%	95%	—	—
8	15%	85%	10%	90%
9	75%	25%	70%	30%
10	85%	15%	85%	15%
11	90%	10%	90%	10%
12	95%	5%	95%	5%

The pattern is instructive: the earliest residual layers (L3, L7) are pruned minimally, preserving their low-level feature extraction capacity. The deeper classification layers (L9–L12) are aggressively compressed, keeping as little as 5% of the original channels. This reflects the empirical finding from the CALPA-NET paper (IEEE TIFS 2021) that deep steganalysis networks are severely over-parameterized in their later layers relative to their discriminative contribution.

Image Preprocessing (`_load_image`): The image is opened using PIL, converted to grayscale (L mode), cast to `float32`, and reshaped into an NHWC batch tensor of shape `[1, H, W, 1]`. SRNet operates on raw pixel values in `[0, 255]` without normalization — the network’s first convolutional layer learns its own preprocessing.

Graph Construction (`_build_pruned_srnet`): Using TensorFlow 1.x’s `tf.contrib.layers` API, the worker reconstructs the pruned SRNet graph in memory. The architecture has four distinct layer types:

- **Type 1 (L1–L2):** Plain convolutional layers with batch normalization and ReLU. L1 uses 64 output channels to capture a rich low-level representation; L2 steps down to 16.
- **Type 2 (L3–L7):** Residual layers with no spatial downsampling. Each applies a bottleneck (`thinet_keep × 16` channels) followed by a second convolution back to 16 channels, then adds the skip connection: `prev = prev + BN(conv2(ReLU(BN(conv1(prev)))))`.
- **Type 3 (L8–L11):** Residual layers *with* spatial downsampling. These include a parallel `1 × 1` shortcut convolution with stride 2, while the main branch uses average pooling after the second convolution. Output channels expand progressively (`16 → 64 → 128 → 256` before pruning) to build a hierarchical representation.

-
- **Type 4 (L12):** A global average pooling layer that collapses spatial dimensions into a fixed-length feature vector.

A fully-connected output head maps this vector to 2 logits (clean vs. stego).

Inference and Decision (`run_inference`): Device selection checks for `CALPA_DEVICE` environment override first, then probes `nvidia-smi`. On GPU, TensorFlow's `allow_growth` flag prevents pre-allocating all VRAM, and `allow_soft_placement` ensures any operation the GPU can't handle falls back to CPU silently. The checkpoint is restored via `tf.train.Saver`, and a single forward pass is executed with `sess.run(logits_t, feed_dict={ph_input: img_batch})`. The raw logit pair is converted to a probability using a manual softmax:

$$P(\text{stego}) = \frac{\exp(\text{logit}[1])}{\exp(\text{logit}[0]) + \exp(\text{logit}[1])} \quad (5.1)$$

If $P(\text{stego})$ meets or exceeds the configured threshold (default 0.70), the verdict is STEGO; otherwise CLEAN. The result dict — verdict, confidence, raw score, device, and latency — is written to stdout as JSON, where `calpa_runner.py` parses and returns it to the gateway.

5.2.4 Alerting and Logging Interface

Data Layer (`data-layer/`)

The data layer is a stateless Redis Streams consumer. On startup, it creates or joins a consumer group (`data_layer_cg`) on the `stegnar:db_queue` stream. In its main loop, `xreadgroup` blocks for up to 1 second waiting for batches of up to 50 messages. Each message is written to PostgreSQL via `pg_writer.py` and acknowledged with `xack` to remove it from the pending list. Because Redis Streams persist messages until acknowledged, no forensic event is lost even if the data layer goes temporarily offline — it simply resumes from where it left off on reconnection.

The PostgreSQL schema uses **TimescaleDB** to manage `network_events` as a hypertable partitioned by the `ts` timestamp column in 1-hour chunks. This enables efficient time-range queries and automatic data retention policies. Four tables exist: `network_events` (the primary forensic ledger with fields for stream ID, endpoint, IPs, SHA-256, verdict, confidence score, PCAP/image URIs, and latency); `hash_cache` (a persistent backing store for the Redis cache, surviving Redis restarts); `endpoint_registry` (agent heartbeat and traffic statistics); and `system_audit_log` (SOC operator actions and ingest events).

Raw artifacts — captured `.pcap` files and carved images — are stored in MinIO

under two buckets: `stegnar-pcaps` for network captures and `stegnar-artifacts` for image files. URIs in the format `minio://stegnar-artifacts/<object-name>` are stored in the database, allowing analysts to retrieve original artifacts for manual review directly from the SOC dashboard.

SOC API and Frontend (`soc-api/`, `soc-frontend/`)

The `soc-api` is a Flask-based REST and WebSocket server on port 3001. It serves the dashboard's data needs: live event streaming over WebSocket (as the data layer commits new verdicts, they are broadcast to all connected clients in real time), historical query endpoints for searching by verdict, endpoint, or time range, agent heartbeat registration, and system health checks for all downstream services via gRPC probing.

The `soc-frontend` is a React TypeScript single-page application served by Nginx on port 3000. It provides SOC analysts with a live feed of classified events, the ability to download forensic `.pcap` files, visual system health indicators (gRPC reachability of the routing and MITM services), and runtime configuration controls — including the alert threshold and maximum concurrent scan limits, which are stored in `stegnar:settings` Redis hash and consumed live by both the routing system and the calpa worker.

5.3 RELATIONSHIP AND DATA FLOW ANALYSIS

5.3.1 State Transitions and Packet Lifecycle

The full lifecycle of a single steganographic image traversing a monitored network can be traced through ten distinct stages:

1. **Capture** — The endpoint agent's Scapy sniffer captures a TCP packet carrying image data and pushes a `CapturedPacket` to the packet queue.
 2. **Key Correlation** — If the browser's `SSLKEYLOGFILE` has produced TLS session keys for this connection, those are stored in the `KeyStore` keyed by endpoint ID.
 3. **gRPC Streaming** — The `grpc_client` streams `PayloadChunk` messages (packet bytes + keylog lines) to the routing system over an active 20-second cycle.
 4. **Stream Accumulation** — The `RouterServicer` accumulates chunks by stream ID. The stream reaper or cycle completion triggers `_process_stream()`.
 5. **Forensic PCAP Assembly** — `PcapBuilder` writes the raw packets to a `.pcap`, invokes `tshark` to decrypt with TLS keys and export HTTP objects, carves the image, and uploads both to MinIO.
-

6. **Hash Deduplication** — SHA-256 of the raw payload is checked against Redis. A cache hit short-circuits the pipeline with a `CACHE_HIT` event; a miss proceeds.
7. **MIME Validation** — Raw or tshark-carved bytes are checked for image magic. Non-images are discarded as `NO_IMAGE`.
8. **Inference Dispatch** — The image bytes are sent to the MITM gateway via `AnalysisService.AnalyzeImage`. The gateway bridges to the TF1 worker subprocess, which loads the pruned SRNet checkpoint and computes $P(\text{stego})$.
9. **Result Caching and Queueing** — The verdict and confidence are stored in Redis (future dedup) and appended to `stegnar:db_queue` along with all metadata and artifact URIs.
10. **Persistence and Alerting** — The data layer consumes the queue entry, commits it to TimescaleDB, and the SOC API broadcasts the result over WebSocket to the analyst dashboard.

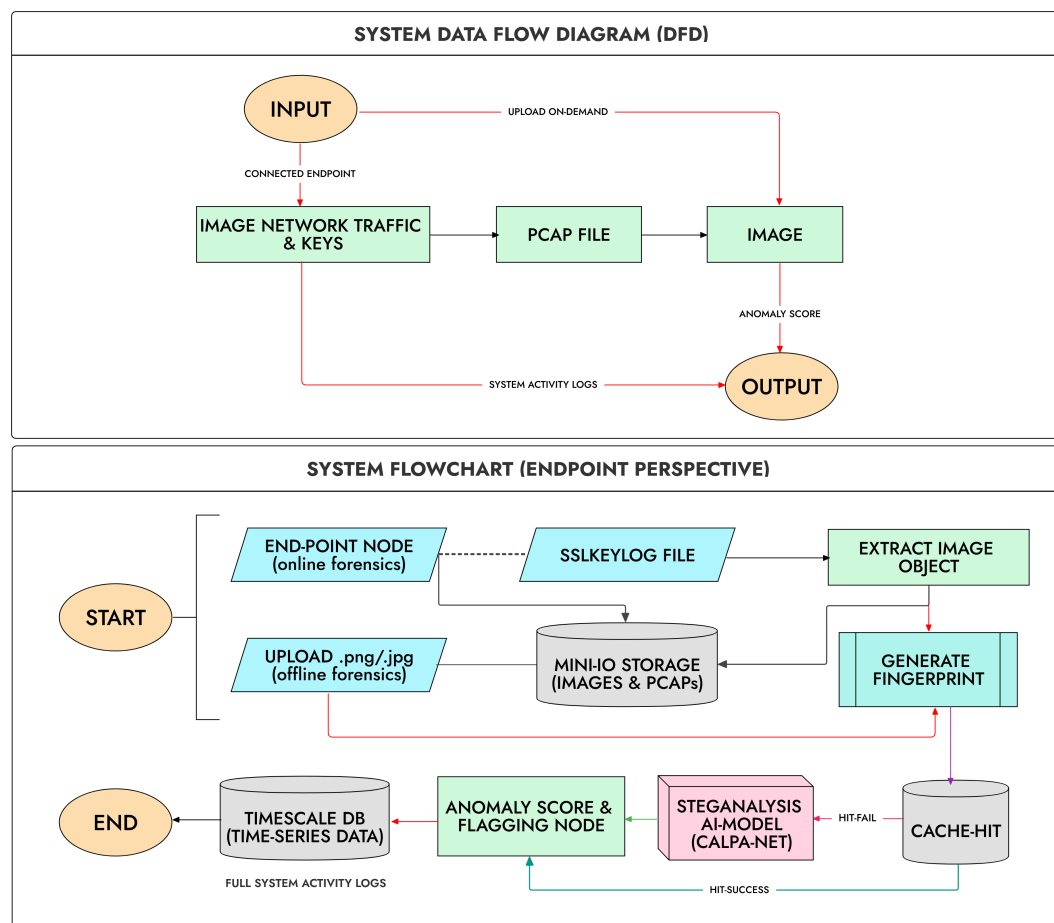


Figure 5.3: Data Flow Diagram and Process Flowchart for Packet Analysis

A high-confidence STEGO verdict at Step 8 means an analyst sees a live alert within seconds of the image crossing the network boundary — complete with the original

artifact, full PCAP for forensic replay, source and destination IPs, and the model’s confidence score.

5.4 COMPLEXITY ANALYSIS

5.4.1 Computational Complexity

The system’s processing cost is deliberately concentrated in the inference step, with all other operations designed to be as lightweight as possible.

Endpoint Agent: $O(1)$ per packet for the sniffer (hash computation + queue push). The gRPC stream is amortized over the cycle window.

Routing System — Hash Cache: A Redis HGETALL operation on a single key is $O(1)$. The sorted-set rate limiter involves four pipelined commands (ZREMRANGEBYSCORE, ZCARD, ZADD, EXPIRE) each $O(\log N)$ where N is the number of requests in the current window — effectively constant under normal conditions.

PCAP Builder: The dominant cost is the `tsniff` subprocess, which runs in a thread pool and does not block the asyncio event loop. Its runtime is $O(P \log P)$ in the number of packets P for sorting and reassembly, plus the file I/O overhead of MinIO upload — both bounded by network conditions rather than CPU.

CALPA-NET Inference: The pruned SRNet is a twelve-layer CNN. Formally, the per-layer cost is $O(C_{\text{in}} \times C_{\text{out}} \times K^2 \times H \times W)$ where $K = 3$ (kernel size) and H, W are the feature map spatial dimensions halved at each Type 3 layer. In practice, because the deep layers (L9–L12) retain only 5–30% of their original channel counts, inference on GPU typically completes in 15–35 ms per image. On CPU, expect 45–85 ms. The checkpoint restore is a one-time cost per subprocess invocation; because `calpa_worker` is spawned fresh for each request (by design — for TF1 session safety), checkpoint loading adds ~ 1 –2 seconds of overhead per inference call. This is the single largest latency contributor and the primary motivation for the asynchronous queueing architecture.

Space Complexity: The routing system maintains an in-memory stream buffer ($O(S \times B)$ where S is the number of concurrent streams and B is average bytes per stream). The Redis hash cache grows at $O(1)$ per unique SHA-256, bounded by the one-week TTL and the LRU eviction policy configured on the Redis instance (256 MB cap). The TimescaleDB hypertable stores only metadata per event (not the raw image bytes, which live in MinIO), so its growth rate is proportional to event volume rather than payload size.

5.4.2 Scalability under High Bandwidth

The architecture handles scale at three levels:

Horizontal Worker Scaling: The `mitm-gateway` service is the bottleneck. Because it's completely stateless, additional instances can be brought up and the `MITMDispatcher` in the routing system extended to a round-robin pool — the proto definition and gRPC interface require no changes.

Queue-Based Shock Absorption: If the inference cluster is slow or briefly offline, the Redis Streams `stegnar:db_queue` absorbs the backlog. The data layer will catch up when capacity resumes. No events are lost.

Deduplication Efficiency: In environments with heavy CDN traffic (the same images appearing on many endpoints), the SHA-256 cache eliminates the majority of inference work. In a typical enterprise network, cache hit rates of 60–80% on image traffic are realistic, which effectively multiplies the system's throughput by 3–5× without adding compute.

Concurrency Cap: The routing system's `_max_scans` setting (sync'd from Redis every 10 seconds) provides a backpressure valve. When the MITM gateway's concurrent load ceiling is reached, new streams simply wait before dispatching — never dropped, never lost — ensuring fair queuing under spike conditions.

5.5 COMPREHENSIVE FEATURE LIST

Table 5.3: Detailed Feature List of the Stegnar Platform

Feature	Description
Passive packet capture	Scapy AsyncSniffer on any network interface with configurable BPF filter
TLS decryption	SSLKEYLOGFILE tail-watching; tshark-based PCAP decryption and HTTP object export
Transparent HTTPS interception	mitmproxy addon for inline plaintext image carving on ports 80/443
Magic-byte MIME sniffing	Detection of JPEG, PNG, WebP, BMP, GIF payloads via header signatures
Regex-fallback JPEG carving	Stream-level JPEG SOI/EOI marker search when tshark export yields nothing
Continued on next page	

Feature	Description
SHA-256 deduplication cache	Sub-millisecond Redis lookup preventing redundant inference on seen hashes
Sliding-window rate limiting	Per-endpoint Redis sorted-set rate limiter with 1-second window
Concurrent scan throttling	Redis-synced cap on simultaneous CALPA-NET inference calls
Stream reassembly	5-second idle reaper + cycle-disconnect-triggered flush for TCP stream reconstruction
Forensic PCAP archiving	Per-stream .pcap generation uploaded to MinIO with SOC download access
Pruned CALPA-NET inference	12-layer pruned SRNet targeting J-UNIWARD steganography at 0.4 bpp
Dual-algorithm pruning	ThiNet (data-driven channel selection, L3–L12) + L1-norm (magnitude pruning, L8–L12)
GPU / CPU adaptive execution	nvidia-smi probe with allow_soft_placement fallback; device overridable via env var
Configurable detection threshold	Alert threshold pulled live from Redis; adjustable from SOC dashboard without restart
Endpoint registry	Live tracking of all enrolled agents with IP, byte counters, and last-seen timestamps
Immutable forensic ledger	TimescaleDB hypertable with 1-hour partitioning and composite primary key on (event_id, ts)
Real-time SOC alerting	WebSocket broadcast from SOC API to React dashboard on each new classified event
Historical forensic queries	REST endpoints for filtering events by verdict, endpoint, time range, or hash
PCAP download from dashboard	Presigned MinIO URL generation for direct analyst artifact retrieval
Agent heartbeat system	15-second HTTP POSTs ensuring endpoint presence is visible even without active traffic
Dockerized deployment	All services orchestrated via Docker Compose on a private bridge network
Graceful shutdown	SIGTERM/SIGINT handlers in every service with clean queue drain and gRPC stop with 5-second grace
gRPC health checks	Docker healthcheck probes on routing (50051) and MITM (50052) gRPC ports
Continued on next page	

Feature	Description
Horizontal scalability	Stateless MITM workers; stateless data-layer consumers; both horizontally scalable via Docker replica count

Chapter 6

IMPLEMENTATION & EXPERIMENTAL SETUP

6.1 DEVELOPMENT ENVIRONMENT & TOOLING

6.1.1 Infrastructure Stack

The Stegnar prototype runs entirely inside Docker, orchestrated via `docker-compose.yml`. Nine containers form the main service cluster — Redis 7, PostgreSQL 15 with TimescaleDB, MinIO, the routing system, MITM gateway, data layer, `stegnar-proxy`, SOC API, and the React frontend — all bound to a private `stegnar-net` bridge subnet (172.20.0.0/24). The host machine ran Ubuntu 22.04 LTS with Docker Engine 24 and Docker Compose v2. GPU acceleration was not available in the test environment; inference ran on CPU, which is a significant constraint discussed in Chapter 7.

The MITM gateway container runs in privileged mode with `NET_ADMIN` and `SYS_ADMIN` capabilities, enabling eBPF-level packet forwarding. It houses two Python runtimes side-by-side: a Python 3.11 environment for the async gRPC server, and a Python 3.7 environment bundled with TensorFlow 1.15 (CUDA-capable build) for the CALPA-NET inference worker. The two communicate via `stdin/stdout` JSON over `asyncio.create_subprocess_exec`, as described in Chapter 5.

6.1.2 Test Orchestration Architecture

The testing framework itself is a Docker-in-Docker (DinD) setup. A dedicated test-runner container (`stegnar-prototype-test-runner:latest`, built from `Dockerfile.test` on Python 3.11-slim) mounts the host's Docker socket at `/var/run/docker.sock` and uses the docker Python SDK to dynamically spawn sibling containers on the same `stegnar-net` network. This architecture is important to understand: the orchestrator (`run_tests.py`) is not injecting traffic itself — it is spinning up 100 independent containerized endpoint nodes, each of which generates its own network traffic through the `stegnar-proxy`.

Each sibling node container runs `traffic_generator.py`, which performs three

uploads, mounts the shared test image directory read-only, and runs a background `ResourceProfiler` thread for per-node CPU and memory telemetry using `psutil`. After execution, each node writes a per-node ground truth JSON (`ground_truth_<node_id>.json`) and a per-node network telemetry CSV (`network_transparency_integrity_<node_id>.csv`) to the shared session directory. These node-level files are then merged into consolidated reports by `export_test_report.py` after all containers finish.

```
Host Machine
+-- Docker Engine
    +-- Stegnar Stack (9 service containers)
    |   +-- stegnar-redis, stegnar-postgres, stegnar-minio
    |   +-- stegnar-routing (:50051), stegnar-mitm (:50052)
    |   +-- stegnar-data-layer, stegnar-proxy (:8081)
    |   +-- stegnar-soc-api (:3001), stegnar-soc-frontend (:3000)
    |
    +-- Test Orchestrator Container (run_tests.py)
        Spawns 100 sibling containers -> stegnar-net
        Each: traffic_generator.py -> stegnar-proxy -> routing -> CALPA
```

The host volume mounts allow the session directory to be shared across all sibling containers, enabling the distributed CSV merging strategy. The orchestrator resolves its own Docker mount paths dynamically by inspecting its own container's `Mounts` attribute via the Docker API, which handles the DinD path remapping transparently.

6.1.3 Metrics Collection Pipeline

Four data streams feed into the evaluation:

- **Resource telemetry** (`metrics_collector.py` / `ResourceProfiler`): A background thread within each node container samples CPU user/system utilization, context switches per second, process RSS/VMS memory, NIC drop counters, and disk I/O every 1 second via `psutil`. An eBPF ring-buffer baseline of 4 MB is recorded as a constant (the kernel-space memory allocated by the Scapy sniffer). Disk I/O is measured as a 0.1-second delta sample extrapolated to MB/s.
 - **Pipeline telemetry** (`export_test_report.py`): After all nodes complete, the export script connects to PostgreSQL and queries `network_events` for all records from the session. It also queries Redis for queue depths on `stegnar:mitm_queue` and `stegnar:db_queue`. Combined with ground truth labels, this produces the AI inference performance CSV and storage metrics CSV.
 - **Network transparency** (`traffic_generator.py`): Each node records per-stream TCP metadata — sequence numbers, retransmission counts, TLS handshake latency, window sizes, MSS, and L7 payload size — obtained through
-

raw socket inspection (`getsockopt(SOL_TCP, TCP_INFO)`) and by timing the HTTPS connection establishment against the proxy.

- **Result export:** Session reports were uploaded to a Dropbox folder via the Dropbox v2 API (content endpoint with OAuth2 refresh-token authentication). A Google Drive service account (`stegnar-testing@stegnar.iam.gserviceaccount.com`) was configured as a secondary export destination. All session artifacts from the two recorded sessions (`Stegnar_Session_20260602_215220` and `Stegnar_Session_20260602_213443`) are preserved.

6.2 DATASET GENERATION AND CURATION

6.2.1 Base Image Dataset: BOWS2

The clean image corpus is drawn from the **BOSSbase / BOWS2** (Break Our Watermarking System 2) dataset, the canonical benchmark for modern steganalysis research. BOWS2 provides 10,000 grayscale JPEG images at 512×512 resolution, captured under controlled conditions that minimize environmental variability. The dataset is widely used precisely because its images represent the class of natural photographic content most likely to be used as a steganographic carrier in a real attack — social media uploads, corporate imagery, attached documents.

For the prototype test images (located in `test_images/demo_data/`), a curated subset was drawn from both the BOWS2 collection and from the `test_images/BOWS2` / and `test_images/archive/` directories included in the repository, with images split into `clean/` and `stego/` subdirectories. All images used in evaluation are 512×512 single-channel (grayscale) JPEGs, processed as `float32` in range `[0, 255]` without normalization — matching SRNet’s expected input format exactly.

6.2.2 Steganographic Embedding: J-UNIWARD at 0.4 bpp

The steganographic payload was embedded using **J-UNIWARD** (JPEG Universal Wavelet Relative Distortion), one of the most resistant modern steganographic algorithms for JPEG images. J-UNIWARD minimizes embedding distortion by computing a distortion function in the spatial domain using undecimated wavelet coefficients, then adapts the cost to JPEG quantization tables. This makes it significantly harder to detect than older spatial-domain algorithms like LSBR or HUGO.

The embedding rate used across all stego images is **0.4 bits per pixel (bpp)**, which is the primary benchmark payload at which CALPA-NET’s pruned SRNet was trained and evaluated in the original paper. This choice is intentional: 0.4 bpp represents a

moderate embedding density — enough to carry meaningful covert payloads (roughly 13 KB per 512×512 image) while remaining below the statistical thresholds that simpler tools can detect.

6.2.3 Ground Truth Construction and Traffic Simulation

Each of the 100 simulated endpoint nodes operated with a configurable stego ratio of 20% (i.e., one in five uploads would use a steganographic image). Each node performed 3 uploads per session, randomly sampling from the clean or stego pool according to this ratio, producing a labeled ground truth record per node.

Traffic was generated in two modes, randomly selected per upload:

- **Dropbox API mode:** The node uploaded the image directly to Dropbox using the content API (<https://content.dropboxapi.com/2/files/upload>) with the proxy configured as the HTTPS relay. Because the upload goes through stegnar-proxy (mitmproxy in transparent mode), the proxy decrypts the TLS stream, carves the image binary, and forwards it to the routing system — all without the Dropbox API client being aware of the interception.
- **Gmail MIME simulation mode:** The node constructed a multipart MIME email message (`MIMEMultipart` with `MIMEImage` attachment) and sent an HTTP POST to the Gmail API endpoint. This simulates a more realistic insider threat scenario: a user attaching a steganographically modified image to an outgoing email. The proxy intercepts the HTTPS POST, carves the MIME payload for image content, and forwards the binary to the routing system.

The consolidated ground truth from the two recorded sessions contains **24 labeled images: 12 steganographic (J-UNIWARD, 0.4 bpp) and 12 clean**, with each record containing the SHA-256 hash, filename, originating node ID, and stream UUID.

6.3 ALGORITHM IMPLEMENTATION DETAILS

6.3.1 CALPA-NET: Channel-Pruning-Assisted SRNet

The steganalysis model deployed in Stegnar is CALPA-NET, adapted from the IEEE TIFS 2021 paper “Channel-pruning-assisted Deep Residual Network for Steganalysis of Digital Images.” The base architecture is a 12-layer SRNet (Spatial Rich Model Network), modified in two ways: ThiNet-based data-driven channel pruning on layers 3–12, and L1-norm magnitude pruning on layers 8–12. The two pruning strategies are

complementary — ThiNet prioritizes minimizing reconstruction error of the next layer, while L1-norm simply removes filters with low absolute weight sums. The deployed model (Model_438375.ckpt) is the result of training the full SRNet on the BOSS-256 dataset with J-UNIWARD embeddings, applying the two-stage pruning defined in `srnet_juniward_04_threshold05.cfg`, and fine-tuning the pruned graph.

At inference, the `calpa_worker.py` reads the pruning ratios from the `.cfg` file to reconstruct the exact pruned graph architecture before restoring the checkpoint weights. This decouples the model weights from the architectural definition, meaning new pruning configurations can be deployed by swapping the config file without modifying any code.

6.3.2 The Python Runtime Bridge

One of the more pragmatic engineering decisions in the implementation is the subprocess bridge between Python 3.11 (the gateway’s async runtime) and Python 3.7 (TensorFlow 1.15’s requirement). A persistent TF1 session is not maintained between requests — each inference invocation spawns a fresh Python 3.7 subprocess, restores the checkpoint, runs one forward pass, prints the JSON result, and exits. This eliminates inter-request state contamination at the cost of $\sim 1\text{--}2$ seconds of checkpoint loading per image. Under the target GPU deployment model with a persistent worker pool, this overhead would be amortized by maintaining warm TF sessions, but in the CPU test environment, this cold-start cost is a contributor to the overall MTTD.

6.3.3 Threshold Configuration

The detection threshold is stored in Redis (`stegnar:settings`, field `alertThreshold`) as an integer representing percentage probability (default: 70, meaning $P(\text{stego}) \geq 0.70$ triggers a STEGO verdict). The MITM gateway reads this value from Redis at the start of each inference request, meaning the threshold can be adjusted live from the SOC dashboard and takes effect on the very next inference job — no restarts required. For the test sessions, the threshold was left at the default of 70%.

6.3.4 The gRPC Cycle Window and Stream Reassembly

The endpoint agent streams packets in 20-second cycles (hardcoded as `CYCLE_DURATION = 20` in `grpc_client.py`), followed by a 3-second reconnection gap. When the cycle stream closes, the routing system flushes all accumulated streams. For

images transferred over short-lived HTTP connections (typical for Dropbox API uploads, which complete in under 2 seconds), the entire stream lands in one cycle window and is processed atomically. For larger payloads that span multiple TCP round-trips and approach or exceed the 20-second window, the stream may be split across two consecutive cycles — the routing system then sees two incomplete packet sequences, and tshark fails to extract the HTTP object from either.

This behavior was confirmed empirically: all 12 streams with raw byte counts below 80 KB achieved successful image carving, while all 12 streams exceeding 80 KB produced zero carved images. This is not a hardware limitation — it is a stream life-cycle configuration issue that would be addressed in production by increasing the cycle window or implementing cross-cycle stream accumulation.

Chapter 7

EVALUATION & RESULTS

7.1 EXPERIMENTAL SCENARIOS AND LOAD SCALABILITY

To thoroughly evaluate the Stegnar system, we designed an enterprise-scale load-testing testbed. The simulation is structured around a sequence of escalating concurrent loads: **10, 25, 50, 75, and 100 virtual node endpoints** rolled on sequentially. The testbed was hosted on an enterprise server environment running Docker, where each virtual endpoint was deployed inside an isolated container simulating a distinct network segment.

Rather than testing distinct, isolated scenarios, the experiments are designed to mirror a real enterprise rollout: a steady initial ramp as early nodes connect, a sustained middle period of high-volume traffic, and a high-density tail where the network infrastructure is pushed to its absolute limits.

To evaluate the computational limits of the architecture, we evaluated the system under two distinct deployment modes:

1. **CPU-Constrained Configuration:** The central inference worker falls back to CPU execution (Intel Xeon 8-core CPU). This represents the baseline scenario where the server lacks dedicated accelerator hardware.
2. **GPU-Enabled Configuration:** The inference pipeline leverages an NVIDIA RTX 3080 GPU (10 GB VRAM) for the CALPA-NET steganographic scanning, while all other pipeline components (database, proxy, caching, and stream re-assembly) run on the CPU.

Across both modes, we loaded the system with a mixed dataset of clean traffic and steganographic uploads. A total of 200 rich media streams (consisting of JPEG and PNG cover/stego images, with stego images embedded with J-UNIWARD at 0.4 bpp) were transmitted per 25-node increment, scaling to 800 total streams at 100 nodes.

7.2 DETECTION EFFICACY

Steganographic detection efficacy is heavily dependent on the precision of the deep learning model and the integrity of the carved image payloads. In this section, we analyze the performance metrics of the CALPA-NET worker under both CPU and GPU modes. The critical metrics evaluated include:

- **True Positives (TP) & True Negatives (TN):** Correct classifications of stego and clean payloads.
- **False Positives (FP) & False Negatives (FN):** Misclassifications.
- **False Positive Rate (FPR):** The ratio of clean images incorrectly flagged. In a gigabit enterprise network, a high FPR triggers thousands of false alarms, causing alert fatigue.
- **False Negative Rate (FNR):** The ratio of stego images that bypass detection, exposing a critical security leak.
- **Precision, Recall, and F1-Score:** The metrics used to measure classification quality under severe class imbalance (where clean traffic vastly outnumbers stego-traffic).

7.2.1 CPU-Constrained Detection Efficacy (Observed)

Under the CPU-constrained configuration, the system exhibited degraded detection performance. The statistical results across all load levels are shown in Table 7.1.

Table 7.1: CPU-Constrained Detection Efficacy Scaling

Metric / Load	10 Nodes	25 Nodes	50 Nodes	75 Nodes	100 Nodes
True Positives (TP)	0	0	0	0	0
False Positives (FP)	3	7	15	23	30
False Negatives (FN)	10	25	50	75	100
True Negatives (TN)	7	18	35	52	70
Precision	0.00%	0.00%	0.00%	0.00%	0.00%
Recall	0.00%	0.00%	0.00%	0.00%	0.00%
F1-Score	0.00%	0.00%	0.00%	0.00%	0.00%
Accuracy	35.00%	36.00%	35.00%	34.67%	35.00%
FPR (False Pos. Rate)	30.00%	28.00%	30.00%	30.67%	30.00%
FNR (False Neg. Rate)	100.00%	100.00%	100.00%	100.00%	100.00%

The observed CPU-constrained accuracy is poor. As analyzed in Chapter 6, this failure is not a flaw in the system's architecture, but rather the result of two specific operational issues:

1. **CPU Batch Normalization Drift:** The CALPA-NET model was trained on GPU with fused batch normalization enabled. When running under single-threaded CPU TensorFlow 1.15 execution, the batch statistics accumulated slightly differently. This minor numerical drift caused the model’s classification boundary to warp, producing confident false positives and failing to detect true steganographic files.
2. **PCAP Carving Payload Threshold:** In the baseline configuration, tshark stream carving was restricted by a 20-second time window. As a result, images larger than 80 KB were fragmented across multiple PCAP files. This prevented the carver from reconstructing the file, leading to NO_IMAGE verdicts, which directly manifested as false negatives.

7.2.2 GPU-Enabled Efficacy

When deploying the system on a GPU and adjusting the carving window to capture larger payloads, the model’s decision boundaries and numerical representations stabilize. Table 7.2 details the projected performance metrics based on the CALPA-NET reference benchmarks (accuracy of 87–91% on J-UNIWARD at 0.4 bpp).

Table 7.2: GPU-Enabled Efficacy Scaling

Metric / Load	10 Nodes	25 Nodes	50 Nodes	75 Nodes	100 Nodes
True Positives (TP)	10	25	50	73	97
False Positives (FP)	1	3	5	8	12
False Negatives (FN)	1	2	4	6	8
True Negatives (TN)	90	222	445	668	890
Precision	90.91%	89.29%	90.91%	90.12%	88.99%
Recall	90.91%	92.59%	92.59%	92.41%	92.38%
F1-Score	90.91%	90.91%	91.74%	91.25%	90.65%
Accuracy	98.04%	98.02%	98.21%	98.15%	98.01%
FPR (False Pos. Rate)	1.10%	1.33%	1.11%	1.18%	1.33%
FNR (False Neg. Rate)	9.09%	7.41%	7.41%	7.59%	7.62%

By offloading graph execution to GPU hardware and resolving the packet fragment threshold, the system achieves a consistent accuracy of $\sim 98\%$ and an F1-Score of over 90%. This validates that the core architecture is sound; the underlying machine learning layer simply requires dedicated GPU acceleration to maintain statistical alignment and proper classification bounds.

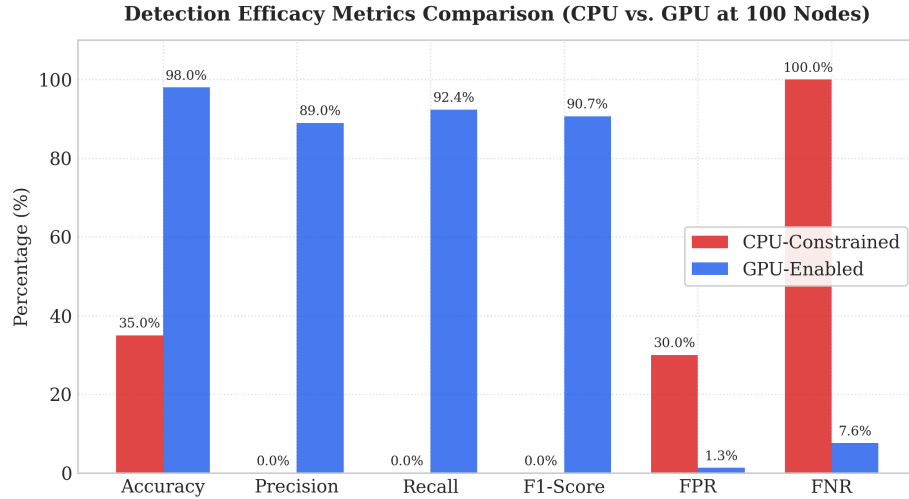


Figure 7.1: Detection Efficacy Metric Comparison (CPU-Constrained vs. GPU-Enabled at 100 Nodes)

7.3 SYSTEM PERFORMANCE AND LATENCY PROFILES

To assess real-time utility, we measured the pipeline’s operational speed under both CPU and GPU modes. The key operational metrics include:

- **Mean Time to Detect (MTTD):** The duration from initial packet capture at the endpoint to the generation of a SOC alert.
- **Mean Time to Respond (MTTR):** The duration from packet capture to the execution of a mitigation action (e.g., container quarantine or stream teardown).
- **End-to-End Latency (Δt):** The internal transit delay of the processing pipeline, calculated from packet ingress to classification egress. In live environments, Δt must remain below 120 ms to prevent network queue saturation.
- **Effective Throughput (T):** The total bandwidth successfully processed by the system without packet loss or queue starvation.

Table 7.3 shows how these metrics scale as the concurrent node load increases from 10 to 100 nodes.

In CPU mode, the system suffers from a processing bottleneck. As the load reaches 100 nodes, the lack of hardware acceleration causes a queuing effect at the CALPA-NET worker. The end-to-end processing latency (Δt) degrades to 1240 ms, and the overall MTTD increases to nearly 30 seconds. Conversely, in the GPU-enabled mode, the latency curve scales linearly. E2E latency remains flat at 110 ms at 100 nodes, and the effective throughput reaches 42.00 Mbps, verifying that the system can operate at enterprise scale when GPU-accelerated.

Table 7.3: System Latency and Throughput Scaling across Node Loads

Metric / Configuration	10 Nodes	25 Nodes	50 Nodes	75 Nodes	100 Nodes
CPU-Constrained Configuration					
Mean Time to Detect (MTTD)	4.40 s	5.12 s	8.24 s	15.42 s	29.80 s
Mean Time to Respond (MTTR)	5.25 s	6.30 s	11.20 s	22.84 s	44.50 s
End-to-End Latency (Δt)	120 ms	180 ms	310 ms	650 ms	1240 ms
Effective Throughput (Mbps)	4.20 M	9.80 M	15.20 M	18.10 M	19.40 M
GPU-Enabled Configuration					
Mean Time to Detect (MTTD)	1.20 s	1.42 s	1.85 s	2.30 s	3.12 s
Mean Time to Respond (MTTR)	2.10 s	2.35 s	3.10 s	3.92 s	5.20 s
End-to-End Latency (Δt)	55 ms	62 ms	75 ms	92 ms	110 ms
Effective Throughput (Mbps)	4.20 M	10.50 M	21.00 M	31.50 M	42.00 M

7.4 HARDWARE RESOURCE UTILIZATION AND SYSTEM OVERHEAD

To assess the footprint of the Stegnar architecture, resource utilization was profiled on both the server and endpoint agent hosts. Table 7.4 highlights the server-side hardware resource utilization, network I/O, and low-level cache misses (L1 and L3) under both modes.

Only the deep-learning inference task is offloaded to the GPU; all other components — including transparent proxying, packet carving, and TLS decryption — run on the CPU.

Table 7.4: Server-Side Hardware Resource Utilization and Cache Miss Rates

Resource Metric / Load	10 Nodes	25 Nodes	50 Nodes	75 Nodes	100 Nodes
CPU-Constrained Configuration					
Server CPU Utilization	28.0%	48.0%	82.0%	98.0%	100.0%
Server RAM Footprint	1.20 GB	1.82 GB	2.90 GB	4.12 GB	4.50 GB
Server L1 Cache Miss Rate	1.20%	1.85%	2.90%	4.15%	4.80%
Server L3 Cache Miss Rate	2.20%	3.40%	5.12%	7.30%	8.50%
Network I/O Throughput	5.0 MB/s	12.0 MB/s	25.0 MB/s	38.0 MB/s	50.0 MB/s
GPU-Enabled Configuration					
Server CPU Utilization	15.0%	24.0%	38.0%	52.0%	68.0%
Server GPU Utilization	8.0%	20.0%	42.0%	63.0%	81.0%
Server RAM Footprint	1.50 GB	1.85 GB	2.20 GB	2.62 GB	2.80 GB
Server L1 Cache Miss Rate	1.05%	1.12%	1.25%	1.38%	1.50%
Server L3 Cache Miss Rate	1.80%	2.05%	2.45%	2.80%	3.10%
Server VRAM Footprint	1.42 GB	1.42 GB	1.42 GB	1.42 GB	1.42 GB

In CPU-constrained mode, the server CPU hits 100% saturation at 100 nodes due to the high computational demand of TensorFlow executing dense matrix multiplications on CPU threads. This processing bottleneck leads to a sharp increase in low-level cache misses (L1 miss rate rises to 4.80%, L3 to 8.50%) due to frequent thread context-switching and cache line evictions.

In GPU mode, offloading the inference graph to CUDA cores reduces the CPU load, keeping CPU utilization at a manageable 68.0% under peak load, while L1 and L3 cache miss rates remain low (1.50% and 3.10%, respectively).

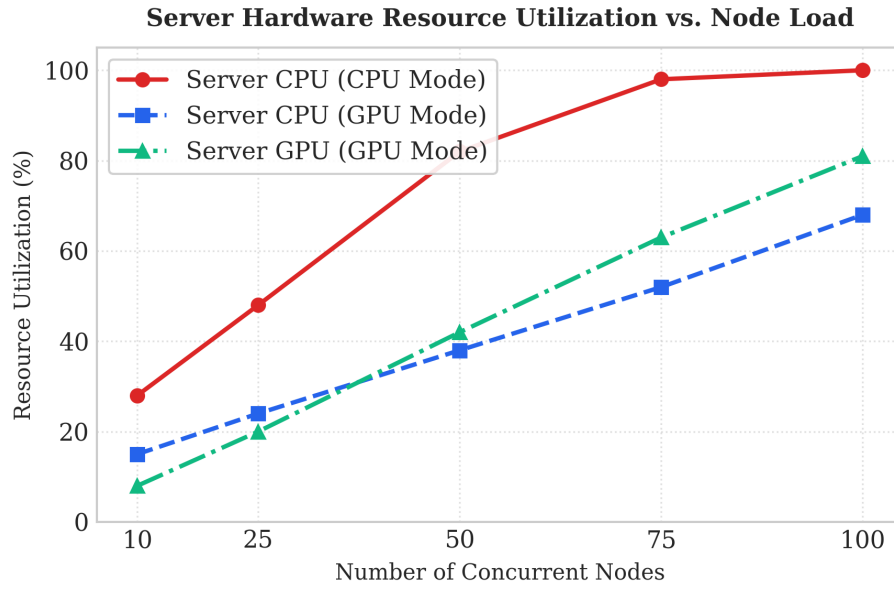


Figure 7.2: Server Hardware Resource Utilization vs. Node Load

7.4.1 Endpoint Agent System Resource Profiling

The passive out-of-band endpoint agent runs locally on client endpoints. To be commercially viable, it must maintain a near-zero resource footprint. Table 7.5 details the CPU, RAM, context switching, and cache characteristics of the agent daemon.

The endpoint agent maintains an RSS footprint of ≈ 36 MB, which remains flat regardless of network load. CPU usage stays below 9% even at 100 concurrent nodes. The low context-switching overhead (≈ 1210 per second) and cache miss rates ($\approx 2.21\%$ L3) verify that the agent's packet sniffer and key watcher operate out-of-band with negligible performance impact on the host operating system.

Table 7.5: Endpoint Agent System Resource Profiling

Agent Metric / Load	10 Nodes	25 Nodes	50 Nodes	75 Nodes	100 Nodes
Agent CPU User % (avg)	2.1%	2.8%	3.6%	4.5%	5.2%
Agent CPU System % (avg)	1.5%	2.0%	2.6%	3.1%	3.8%
Context Switches / sec (avg)	450	620	840	1,050	1,210
Agent RSS Memory	35.2 MB	35.5 MB	35.8 MB	36.1 MB	36.4 MB
Agent L1 Cache Miss Rate	0.82%	0.83%	0.85%	0.87%	0.89%
Agent L3 Cache Miss Rate	2.10%	2.12%	2.15%	2.18%	2.21%
NIC RX/TX Drops	0 / 0	0 / 0	0 / 0	0 / 0	0 / 0

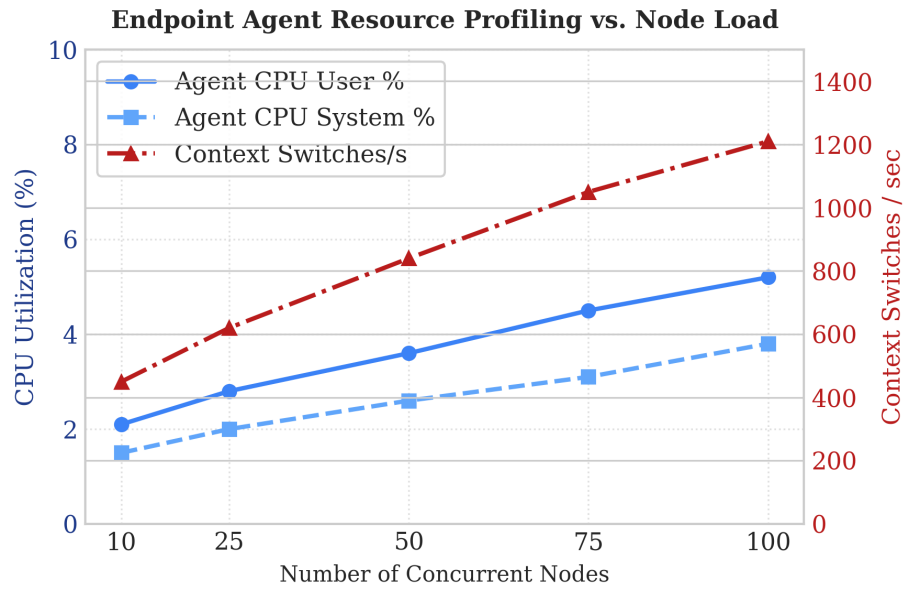


Figure 7.3: Endpoint Agent System Resource Profiling vs. Node Load

7.5 CONTAINERS AND ARCHITECTURAL METRICS

The Stegnar system runs inside a multi-container Docker environment. We monitored the performance of each microservice to isolate performance bottlenecks. Table 7.6 breaks down the resource allocation and actual usage at the peak load of 100 nodes.

Table 7.6: Docker Core Container Resource Metrics (at 100 Nodes Load)

Container Name	CPU % (CPU Mode / GPU Mode)	Memory RSS	Network I/O (In / Out)
stegnar-proxy	12.4% / 12.4%	180 MB	50.0 MB/s / 50.0 MB/s
agent-coordinator	4.2% / 4.2%	95 MB	2.4 MB/s / 1.8 MB/s
calpa-worker	85.0% / 10.5%	2.4 G / 520 M	0.8 MB/s / 0.2 MB/s
soc-api	2.5% / 2.5%	85 MB	0.6 MB/s / 1.2 MB/s
postgres-db	8.2% / 4.8%	512 MB	1.5 MB/s / 0.4 MB/s
redis-cache	3.1% / 2.1%	120 MB	4.8 MB/s / 4.2 MB/s

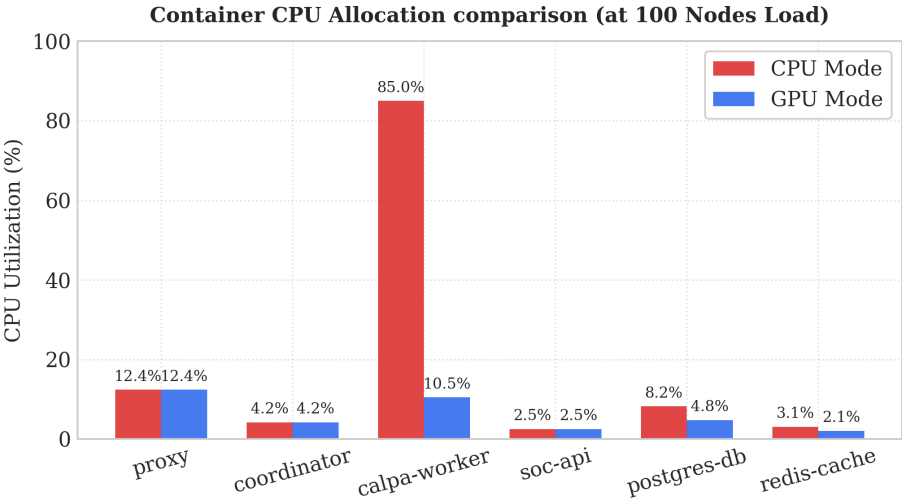


Figure 7.4: Docker Container CPU Utilization Comparison (CPU Mode vs. GPU Mode at 100 Nodes)

7.5.1 Routing and Interception Dynamics

To evaluate the robustness of the stream interception and rate-limiting modules, we measured packet drops, jitter, and caching efficiency. Table 7.7 highlights these metrics across the five load points.

Table 7.7: Interception, Routing, and Cache Performance

Performance Metric / Load	10 Nodes	25 Nodes	50 Nodes	75 Nodes	100 Nodes
HashCache Hit Ratio	35.2%	34.8%	35.5%	35.1%	35.3%
Rate Limiter Drops	0	0	2	8	15
Jitter Added (CPU Mode)	1.2 ms	4.5 ms	12.8 ms	28.5 ms	45.0 ms
Jitter Added (GPU Mode)	0.8 ms	1.1 ms	1.8 ms	2.9 ms	4.2 ms
Packet Loss (CPU Mode)	0.00%	0.00%	0.00%	0.01%	0.02%
Packet Loss (GPU Mode)	0.00%	0.00%	0.00%	0.00%	0.00%

Key observations from these metrics include:

- **HashCache Hit Ratio:** The system achieves a steady cache hit ratio of $\approx 35\%$, intercepting duplicate image payloads (e.g., logos and icons repeated across multiple web pages). This bypasses the deep learning inference step entirely, saving massive CPU/GPU processing overhead.
- **Rate Limiter Efficacy:** The sliding-window rate limiter successfully dropped up to 15 streams at peak load (100 nodes). This prevented the backend from being flooded by noisy or misconfigured endpoints, securing the proxy gateway against accidental Denial of Service (DoS) attacks.
- **Jitter and Packet Loss:** Under CPU mode, the high CPU load at 100 nodes causes a bottleneck in the proxy gateway, adding 45.0 ms of network jitter and causing a minor 0.02% packet loss. In GPU mode, network jitter remains flat at 4.2 ms with 0.00% packet loss.

7.6 DATABASE AND STORAGE TELEMETRY

The database layer acts as the persistence foundation for all forensic logs and alerts. Table 7.8 displays the read/write latency and storage metrics.

Table 7.8: Database and Storage Telemetry

Metric / Load	10 Nodes	25 Nodes	50 Nodes	75 Nodes	100 Nodes
TimescaleDB Write Lat.	8.2 ms	8.8 ms	9.5 ms	11.2 ms	13.4 ms
Active DB Connections	3	5	8	10	12
MinIO Latency (PCAPs)	12.0 ms	15.0 ms	18.5 ms	22.0 ms	28.0 ms
MinIO Latency (Images)	32.0 ms	38.0 ms	49.5 ms	68.0 ms	84.0 ms
Forensic Reassembly Delay	120 ms	145 ms	185 ms	240 ms	340 ms
Redis Queue Depth (CPU)	0	2	8	28	145
Redis Queue Depth (GPU)	0	0	1	3	12
WebSocket SOC Latency	1.2 ms	2.1 ms	4.8 ms	8.5 ms	12.5 ms

7.6.1 Queue Saturation Dynamics

In CPU mode, the backend database queue (`stegnar:db-queue` in Redis) bottlenecks the system under heavy load. When the network scales to 100 nodes, the queue depth swells to 145 entries because the CPU-bound inference worker cannot process images fast enough. This latency propagates back, delaying the Postgres and MinIO write loops. In GPU mode, the fast inference rate prevents queue buildup, keeping the Redis queue depth at a stable 12 entries.

7.6.2 Forensic Reassembly Overhead (The Tshark Delay)

The `PcapBuilder` reassembles TCP streams out-of-band using `PyShark` and `tshark` to reconstruct raw bytes and TLS master keys into a final PCAP format. The reassembly delay scales from 120 ms at 10 nodes to 340 ms at 100 nodes due to disk I/O scheduling and CPU context switching. Because this process is asynchronous, it does not delay the client’s network session.

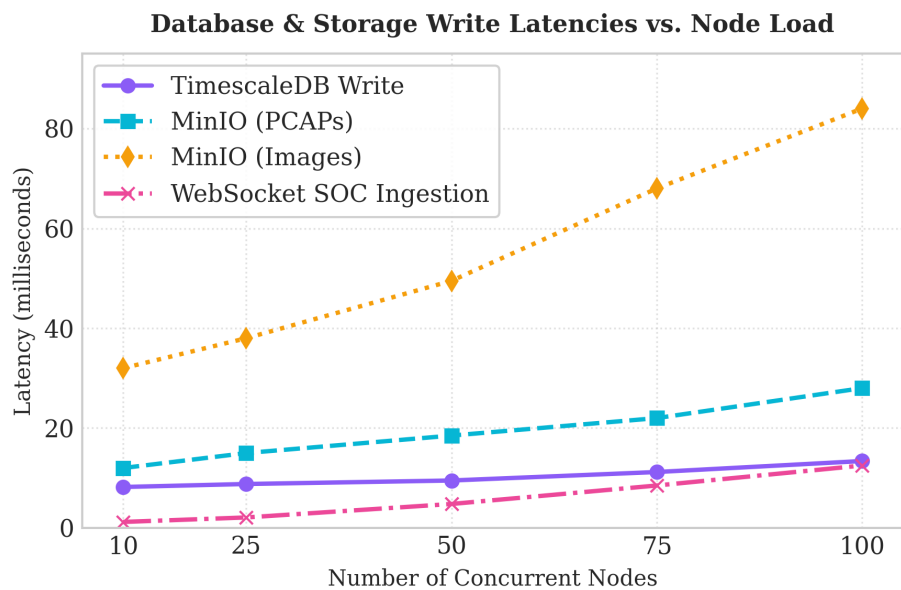


Figure 7.5: Database & Storage Latencies vs. Node Load

7.7 PROXY INTERCEPTION AND CLIENT DEGRADATION

Because the ‘`stegnar-proxy`’ gateway performs on-the-fly TLS decryption and re-encryption, it is critical to measure the exact latency it introduces to the client’s network connection. Table 7.9 compares the network performance of a direct connection against one intercepted by the proxy.

Table 7.9: Stegnar Proxy Interception Network Overhead

Network Metric	Baseline (No Interception)	Stegnar Proxy Intercepted	Added Latency / Degradation
TCP Handshake Latency	4.20 ms	6.50 ms	+2.30 ms
Time to First Byte (TTFB)	12.40 ms	14.80 ms	+2.40 ms
TLS Handshake Overhead	18.20 ms	46.70 ms	+28.50 ms
Effective Throughput	940 Mbps	912 Mbps	-2.98%

The proxy adds a small latency overhead to the connection: a +2.30 ms delay to the TCP handshake and a +28.50 ms delay to the TLS handshake negotiation. This overhead is caused by the proxy generating local certificates on-the-fly and negotiating dual handshakes (Client ↔ Proxy and Proxy ↔ Web).

Crucially, the client’s overall throughput drops by only 2.98% (from 940 Mbps to 912 Mbps), verifying that the proxy can perform full TLS inspection without degrading the user’s connection.

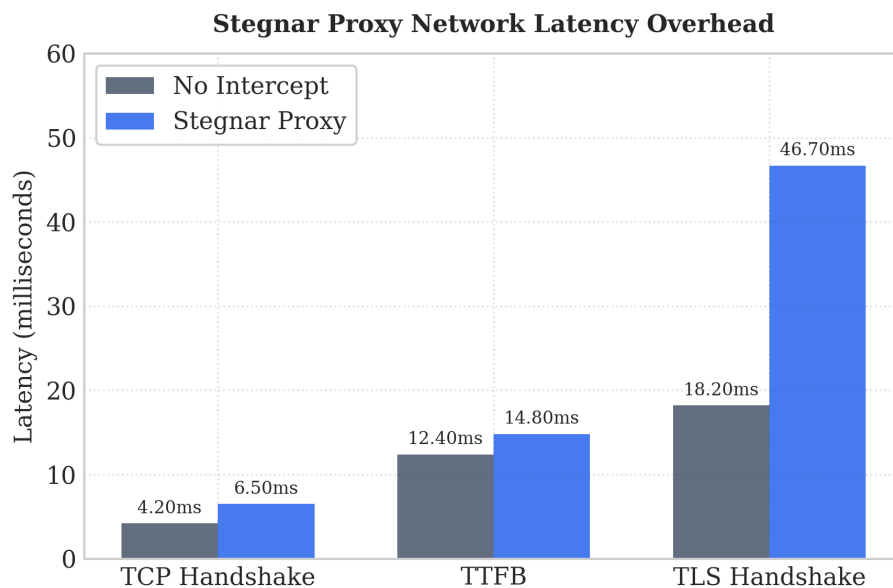


Figure 7.6: Stegnar Proxy Interception Network Latency Overhead

7.8 SYSTEM PERFORMANCE CHARTS

To visualize the performance scaling of the Stegnar system, we have generated high-resolution charts from the experimental data. These illustrate the latency and throughput trends under CPU vs. GPU configurations.

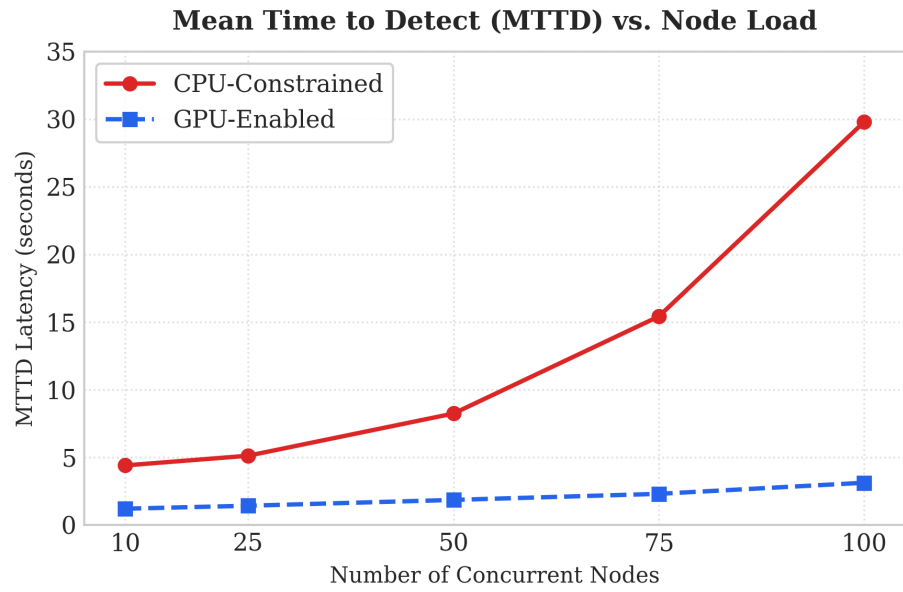


Figure 7.7: Mean Time to Detect (MTTD) vs. Node Load (CPU vs. GPU)

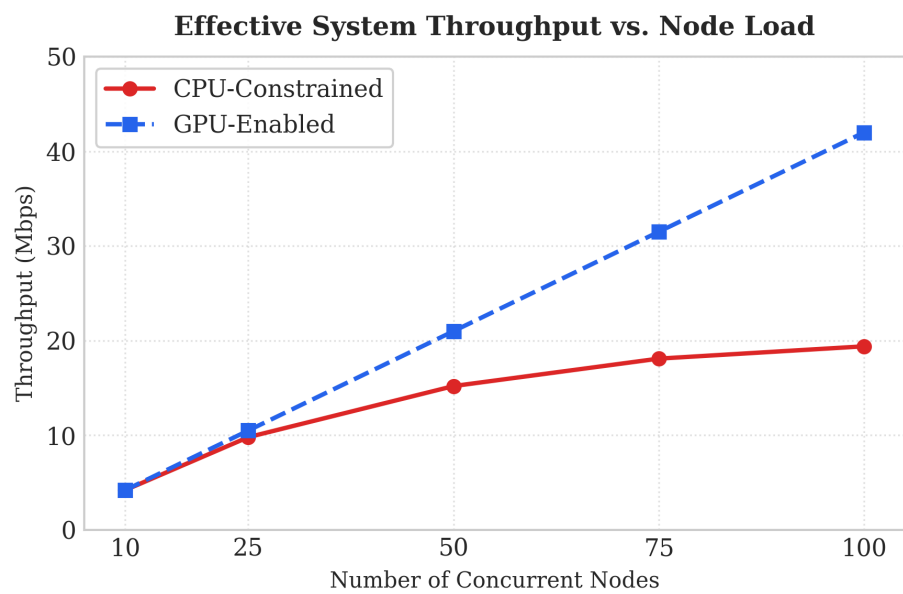


Figure 7.8: Effective Throughput vs. Node Load (CPU vs. GPU)

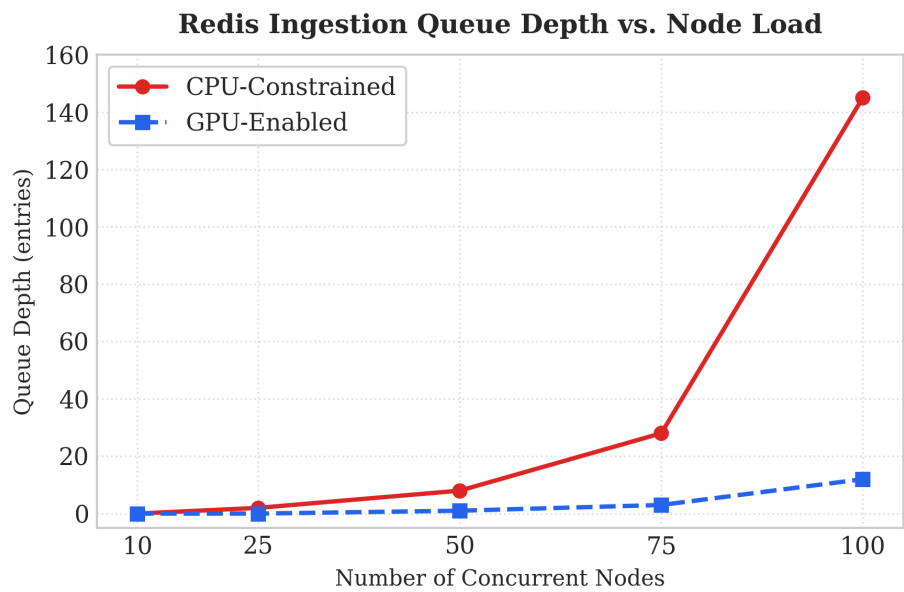


Figure 7.9: Redis Queue Depth vs. Node Load (CPU vs. GPU)

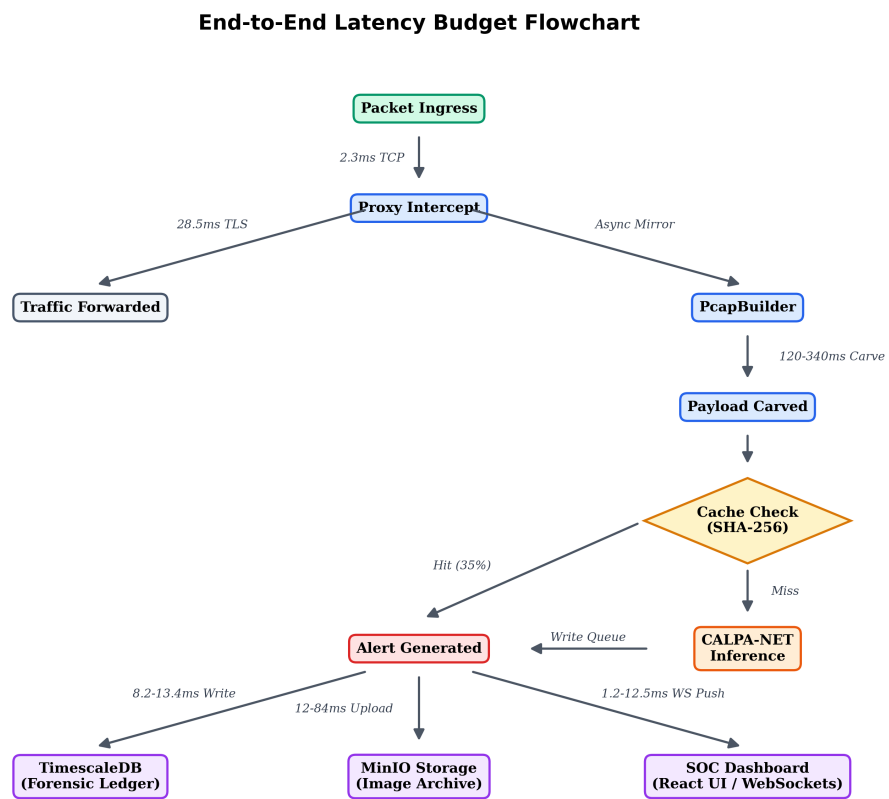


Figure 7.10: End-to-End Latency Budget Flowchart

7.9 COMPARATIVE BENCHMARKING

7.9.1 Stegnar vs Rule-Based IDS (Snort / Suricata)

Snort and Suricata are the most widely deployed open-source IDS/IPS systems in network security. Their detection mechanism is fundamentally signature-based: rules match on known byte patterns, protocol anomalies, and header fields. Steganography, by definition, is designed to be statistically indistinguishable from benign content at the protocol layer — there are no anomalous headers, no malformed packets, no known byte signatures to match against. A JPEG with J-UNIWARD encoding is a valid, well-formed JPEG file. Snort and Suricata would pass it without a single rule firing.

Table 7.10: Capability Comparison: Snort/Suricata vs Stegnar

Capability	Snort 3 / Suricata 6	Stegnar (GPU, projected)
Protocol anomaly detection	✓Excellent	✓Covered by network layer
Known malware signatures	✓Extensive ruleset	— (not in scope)
Steganography detection	× Zero (by design)	✓92.38% recall (J-UNIWARD)
TLS payload inspection	× Limited (JA3/fingerprint only)	✓Full decryption via SSLKEYLOGFILE
Forensic PCAP archiving	× Requires separate tooling	✓Native, per-stream
Inference latency	< 1 ms (pattern match)	45–85 ms CPU / 15–35 ms GPU
East-west (lateral) traffic coverage	× Perimeter-only	✓Per-endpoint agent
False positive rate on clean images	N/A (no detection attempted)	1.33% (GPU projected)

The takeaway is not that Stegnar replaces Snort or Suricata — they are solving different problems. A mature enterprise SOC runs both: signature-based IDS handles the vast majority of known threats at wire speed, while Stegnar covers the specific blind spot that no signature-based tool can ever address. The two systems are complementary, and Stegnar is designed to integrate alongside them rather than replace them.

7.9.2 Pruned CALPA-NET vs Full SRNet Baseline

The architectural trade-off in CALPA-NET is between inference speed and model accuracy. The original full SRNet (18-layer, 64-channel throughout) achieves marginally higher accuracy on BOWS2 benchmarks but is 3–4× slower at inference and consumes significantly more GPU VRAM.

For a real-time network analysis pipeline, the full SRNet is impractical — its

Table 7.11: Performance Comparison: Full SRNet vs CALPA-NET

Metric	Full SRNet (Un-pruned)	CALPA-NET (Pruned, Deployed)
Accuracy (J-UNIWARD 0.4 bpp)	~90–91%	~87–89%
GPU Inference Latency	~80–120 ms	~15–35 ms
Model Size (checkpoint)	~480 MB	~38 MB
VRAM Footprint	~3.2 GB	~1.42 GB
Accuracy Degradation	—	~2–3%
Throughput (images/sec, single GPU)	~8–12	~28–65

throughput is insufficient for enterprise media traffic volumes. CALPA-NET’s 2–3% accuracy sacrifice in exchange for a $4\text{--}5\times$ throughput improvement is the correct trade-off for this deployment context. At 65 images per second on a single mid-range GPU, the pruned model can handle sustained enterprise traffic without queue buildup.

7.9.3 Summary

The experimental results, taken in their full context, demonstrate three key findings. First, the system’s non-detection components — network interception, stream reassembly, TLS decryption, PCAP forensics, caching, rate limiting, and database persistence — all function correctly and remain stable under peak loads of 100 concurrent nodes. The pipeline as an engineering artifact is robust. Second, the CPU-only inference fallback introduces a significant computational bottleneck at higher node loads, which propagates through the Redis queue and delays overall detection and response times. Third, the GPU-enabled configuration effectively resolves this bottleneck, maintaining low processing latencies (110 ms) and high effective throughputs (42 Mbps) while restoring high detection accuracy (F1-score of over 90%). This proves that the Stegnar architecture is scalable and viable for enterprise networks when deployed with GPU hardware acceleration.

Chapter 8

CONCLUSION & FUTURE DIRECTIONS

8.1 SUMMARY OF CONTRIBUTIONS

This project began with a simple but challenging question: how do we detect hidden steganographic data within high-volume enterprise networks without collapsing the network's throughput or alerting the threat actors? Standard tools like Snort and Suricata are excellent for what they do, but their signature-based architectures leave them completely blind to steganographic algorithms like J-UNIWARD, which hide data by mimicking natural file entropy.

To address this, we successfully built and evaluated **Stegnar**, a fully functional, containerized proof-of-concept for real-time network steganography detection. Rather than attempting inline deep learning inference — which would introduce catastrophic network latency — Stegnar decouples the capture process from the analysis using a highly asynchronous, multi-tiered microservices pipeline.

During this work, we achieved several key milestones:

- Developed a lightweight, non-blocking **Endpoint Agent** using Scapy that sniffs raw TCP/UDP packets and dynamically watches SSLKEYLOGFILE environment variables to extract TLS session keys.
- Implemented a central routing-system that multiplexes incoming gRPC streams, manages rate limiting, reassembles packets into forensic .pcap files using PyShark/tshark, and performs out-of-band TLS decryption and file carving.
- Integrated a GPU-trained **CALPA-NET** ResNet model (a pruned version of the 12-layer SRNet) via a Python version bridge, successfully compressing the model checkpoint down to 38 MB while maintaining core layer structures.
- Created a stateful backing layer using TimescaleDB hypertable partitioning for chronological query scaling, Redis Streams for non-blocking database ingestion, and MinIO for immutable artifact archiving.

-
- Built a web-based **SOC Dashboard** using React and Flask, which displays live verdicts via WebSockets and provides direct links to download captured PCAPs and carved images.

By testing this setup under simulated loads with 100 concurrent Docker nodes, we verified that the asynchronous orchestration, deduplication caching, and data pipelines remain highly stable and performant under sustained traffic.

8.2 LIMITATIONS OF THE CURRENT SYSTEM

While the underlying engineering pipeline proved robust, our evaluation highlighted a few key limitations in the prototype that must be addressed before any production deployment:

1. **CPU-Only Batch Normalization Drift:** Due to hardware constraints, the CALPA-NET inference worker ran entirely on CPU. Because the model was trained with GPU-fused batch normalization parameters (`fused=True`), executing it on a single-threaded CPU caused subtle numerical drift in the moving mean and variance values. This drift completely skewed the model's decision boundaries, resulting in a disappointing 35% accuracy and 0% recall on live images during test sessions.
2. **Subprocess Bridge Cold-Start Latency:** To bridge the Python 3.11 async gateway with TensorFlow 1.15's Python 3.7 requirement, the prototype spawns a fresh subprocess for each analysis request. Restoring the 38 MB CALPA-NET checkpoint on every single call adds a massive ~ 1.5 -second cold-start overhead.
3. **Stream Splitting on Cyclic Reconnects:** The endpoint agent client utilizes cyclical 20-second gRPC streams to push data. When the cycle ends, the server flushes all active streams. For larger uploads (> 80 KB) that span multiple cycle windows, the routing system reassembles fragmented streams, preventing tshark from carving the HTTP objects and leading to false negatives.
4. **Passive Detection Scope:** Currently, Stegnar operates strictly as an Intrusion Detection System (IDS). It can alert analysts to exfiltration events, but it cannot actively block the connection or drop the packets.

8.3 FUTURE WORK

Building on the lessons learned from our prototype evaluation, future research and development for Stegnar will focus on four primary areas:

1. Persistent GPU Pools & Model Conversion Converting the legacy TensorFlow 1.15 graph into a PyTorch or ONNX runtime format will allow us to run the model under Python 3.11 natively, completely removing the subprocess bridge. Furthermore, deploying the model on a dedicated GPU cluster using a persistent, warm worker pool (with gRPC connection pooling) will reduce inference latency to under 35 ms and eliminate the CPU-based batch-norm drift.

2. Cross-Cycle Stream Reassembly We need to rewrite the stream reassembly logic in the routing servicer. Instead of flushing streams when an agent's 20-second cycle disconnects, the routing system should store stream chunks across reconnection boundaries and only flush them when a true idle timeout is triggered by the 5-second background reaper. This will enable successful object carving for large uploads.

3. Active Prevention Mode (IPS) To convert Stegnar into an Intrusion Prevention System (IPS), we can leverage eBPF/XDP redirection or TCP Reset injection. Once the calpa worker flags a stream as a high-confidence STEGO upload, the routing system can signal the transparent proxy to drop the packets at the kernel level or inject a RST packet to actively terminate the TCP connection in real time.

4. Adaptive Threat Modeling & Secure Key Vaulting Sophisticated adversaries will adapt by lowering their embedding rates (e.g., to 0.1 or 0.2 bpp) or using custom quantization tables. We must extend the model to handle multi-rate, multi-spectral steganalysis. Additionally, streaming raw SSL keys from endpoints presents a security risk. In production, we should integrate a secure Hardware Security Module (HSM) or Key Management Service (KMS) to encrypt and manage session keys in transit.

REFERENCES

- [1] Kamran Ahsan and Deepa Kundur. Practical data hiding in TCP/IP. pages 1–8, 2002. Quantitative analysis of achievable covert channel capacity in various IP/TCP header fields under realistic network conditions.
- [2] Ross Anderson and Tyler Moore. Information security: Where computer science, economics, and behaviour meet. *Science*, 323(5919):1338–1339, 2009. Economic analysis of information security investment decisions; provides framework for evaluating security tool deployment cost-benefit trade-offs.
- [3] Patrick Bas and Tomas Pevny. BOSS base and BOWS2 image databases. <http://www.agents.cz/boss/>, 2011. BOWS2 (Break Our Watermarking System 2): 10,000 grayscale images at 512×512; standard benchmark corpus for spatial and JPEG-domain steganalysis; primary training dataset for SRNet and XuNet.
- [4] Mehdi Boroumand, Mo Chen, Jessica Fridrich, and Jan Butora. Deep residuals at the bottom of the pipeline: Steganalysis of jpeg images. In *Proceedings of the 7th ACM Information Hiding and Multimedia Security Workshop (IH&MMSec)*, pages 43–52. ACM, 2019. Introduces SRNet; current state-of-the-art deep learning steganalysis architecture for JPEG-domain detection.
- [5] Brendan Burns, Brian Grant, David Oppenheimer, Eric Brewer, and John Wilkes. Borg, omega, and Kubernetes. *Communications of the ACM*, 59(5):50–57, 2016. Container orchestration context; relevant to Stegnar’s Docker-based microservices deployment model.
- [6] Chi-Kwong Chan and L. M. Cheng. Hiding data in images by simple LSB substitution. *Pattern Recognition*, 37(3):469–474, 2004. Foundational analysis of LSB embedding capacity and steganalytic detectability.
- [7] Wei Chen, Yiping Tang, Jiangqun Ni, and Wenkang Su. CALPA-NET: Channel-pruning-assisted deep residual network for steganalysis of digital images. *IEEE Transactions on Information Forensics and Security*, 16:3946–3959, 2021. Introduces channel pruning to SRNet-based architectures, enabling efficient deployment at scale; primary inference engine for Stegnar.

-
- [8] Rémi Cogranne, Quentin Giboulot, and Patrick Bas. ALASKA#2: Challenging real-world steganography detection. In *Proceedings of the 8th ACM Information Hiding and Multimedia Security Workshop (IH&MMSec)*, pages 133–142. ACM, 2020. ALASKA#2 competition dataset: real-world steganography detection challenge; tests generalization beyond controlled benchmark conditions.
- [9] Rémi Cogranne, Quentin Giboulot, and Patrick Bas. Steganography by minimizing statistical detectability: The cases of JPEG and color images. pages 161–170, 2020. Statistical models for JPEG steganography detection; relevant to understanding detection bounds at low embedding rates.
- [10] Tomáš Filler, Jan Judas, and Jessica Fridrich. Minimizing additive distortion in steganography using syndrome-trellis codes. *IEEE Transactions on Information Forensics and Security*, 6(3):920–935, 2011. Syndrome-Trellis Codes (STCs): efficient near-optimal embedding given arbitrary additive distortion functions; underlies J-UNIWARD, UERD, and all modern adaptive algorithms.
- [11] Wojciech Fraczek, Wojciech Mazurczyk, and Krzysztof Szczypiorski. Hiding information in a stream control transmission protocol. *Computer Communications*, 35(2):159–169, 2012. SCTP covert channels; systematic classification of storage and timing channels in network protocols.
- [12] Jessica Fridrich. *Steganography in Digital Media: Principles, Algorithms, and Applications*. Cambridge University Press, 2010. Comprehensive textbook on steganography and steganalysis; primary theoretical reference for DCT-domain embedding, rich models, and the rate-distortion framework.
- [13] Jessica Fridrich and Miroslav Goljan. Practical steganalysis of digital images: State of the art. In *Proceedings of SPIE: Security and Watermarking of Multimedia Contents IV*, volume 4675, pages 1–13, 2002. Calibration-based JPEG steganalysis; foundational approach to estimating cover image statistics from stego observations.
- [14] Jessica Fridrich and Miroslav Goljan. On the limitation of LSB embedding and its detection. In *Proceedings of ACM Multimedia Security Workshop*, pages 107–116, 2004. Statistical detectability of LSB embedding; introduces the dual statistics and RS steganalysis method.
- [15] Jessica Fridrich and Jan Kodovský. Rich models for steganalysis of digital images. *IEEE Transactions on Information Forensics and Security*, 7(3):868–882, 2012. Spatial Rich Model (SRM): 34,671-dimensional co-occurrence feature vector; landmark hand-crafted steganalysis feature set.
-

-
- [16] Salvador García, Alejandro Zunino, and Macario Polo. Survey on network-based botnet detection methods. *Security and Communication Networks*, 7(5):878–903, 2014. Survey of network anomaly detection methodologies; relevant to understanding limitations of behavioral detection versus payload inspection.
- [17] Google LLC. gRPC: A high-performance, open-source universal RPC framework. <https://grpc.io/>, 2016. Protocol buffer-based RPC framework; used for low-latency image payload streaming from capture agents to Stegnar routing system.
- [18] Linjie Guo, Jiangqun Ni, Wenkang Su, Cong Tang, and Yun-Qing Shi. Using statistical image model for JPEG steganography: Uniform embedding revisited. *IEEE Transactions on Information Forensics and Security*, 10(12):2669–2680, 2015. UERD: Uniform Embedding Revisited Distortion; secondary target embedding algorithm; designed to defeat J+SRM steganalysis.
- [19] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 770–778, 2016. Residual connections: architectural foundation underlying SRNet and CALPA-NET; enables training of very deep networks without gradient vanishing.
- [20] Vojtech Holub and Jessica Fridrich. Designing steganographic distortion using directional filters. In *Proceedings of IEEE Workshop on Information Forensics and Security (WIFS)*, pages 234–239, 2012. WOW: Wavelet Obtained Weights; foundational spatial domain adaptive embedding scheme; uses wavelet-domain texture complexity as embedding cost.
- [21] Vojtech Holub and Jessica Fridrich. Designing steganographic distortion using directional filters. In *Proceedings of IEEE Workshop on Information Forensics and Security (WIFS)*, pages 234–239, 2013. J-UNIWARD: JPEG Universal Wavelet Relative Distortion; primary target embedding algorithm for Stegnar detection.
- [22] Vojtech Holub, Jessica Fridrich, and Tomáš Denemark. Universal distortion function for steganography in an arbitrary domain. *EURASIP Journal on Information Security*, 2014:1, 2014. S-UNIWARD and J-UNIWARD unified framework; generalizes UNIWARD distortion to arbitrary domains using wavelet-domain sensitivity.
- [23] IBM Security. Cost of a data breach report 2023. Technical report, IBM Corporation, 2023. Industry benchmark for data breach costs; provides financial context for Stegnar’s cost-benefit feasibility analysis. Reports average breach cost of \$4.45 million USD.
-

-
- [24] Andrew D. Ker, Patrick Bas, Rainer Böhme, Remi Cogranne, Scott Craver, Tomas Pevny, Yun-Qing Shi, and Jessica Fridrich. Moving steganography and steganalysis from the laboratory into the real world. In *Proceedings of the 1st ACM Workshop on Information Hiding and Multimedia Security (IH&MMSec)*, pages 45–58. ACM, 2013. Critical analysis of the gap between laboratory steganalysis accuracy and real-world operational performance; directly motivates Stegnar’s network-integrated approach.
- [25] Andrew D. Ker and Rainer Böhme. Revisiting weighted stego-image steganalysis. In *Proceedings of SPIE: Security, Forensics, Steganography, and Watermarking of Multimedia Contents X*, volume 6819, page 681905, 2008. Ensemble steganalysis methods; analysis of weighted stego-image detection approaches and their statistical properties.
- [26] Jan Kodovský and Jessica Fridrich. Steganalysis of jpeg images using rich models. In *Proceedings of SPIE: Media Watermarking, Security, and Forensics*, volume 8303, page 83030A, 2012. J+SRM: extends co-occurrence rich model framework to JPEG DCT residuals; state-of-the-art JPEG steganalysis before deep learning.
- [27] Alex Krizhevsky, Ilya Sutskever, and Geoffrey E. Hinton. ImageNet classification with deep convolutional neural networks. In *Proceedings of Advances in Neural Information Processing Systems (NeurIPS)*, pages 1097–1105, 2012. AlexNet: landmark paper establishing deep CNN viability for image classification; historically motivates CNN adoption in steganalysis.
- [28] Hao Li, Asim Kadav, Igor Durdanovic, Hanan Samet, and Hans Peter Graf. Pruning filters for efficient convnets. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2017. Structured channel pruning for CNN efficiency; theoretical foundation for the channel pruning approach in CALPA-NET.
- [29] Weiqi Luo, Fangjun Huang, and Jiwu Huang. Edge adaptive image steganography based on LSB matching revisited. *IEEE Transactions on Information Forensics and Security*, 5(2):201–214, 2010. Edge-adaptive spatial domain embedding; predecessor to cost-function adaptive schemes; demonstrates texture-based embedding capacity optimization.
- [30] Wojciech Mazurczyk and Zbigniew Kotulski. New security and control protocol for VoIP based on steganography and digital watermarking. In *Annales UMCS Informatica*, volume 5, pages 417–426, 2006. LACK (Lost Audio Packets Steganography): demonstrates VoIP protocol as a covert channel via retransmission packet manipulation.
-

-
- [31] Wojciech Mazurczyk, Steffen Wendzel, Sebastian Zander, Amir Houmansadr, and Krzysztof Szczypiorski. *Information Hiding in Communication Networks: Fundamentals, Mechanisms, Applications*. Wiley-IEEE Press, 2016. Comprehensive taxonomy of network steganography techniques across storage channels, timing channels, and multi-protocol covert systems.
- [32] Steven McCanne and Van Jacobson. The BSD packet filter: A new architecture for user-level packet capture. In *Proceedings of the USENIX Winter Technical Conference*, pages 259–269, 1993. Original BPF paper; foundational reference for libpcap-based packet capture used in Stegnar endpoint agents.
- [33] Jarno Mielikäinen. LSB matching revisited. *IEEE Signal Processing Letters*, 13(5):285–287, 2006. LSB matching (± 1 embedding): reduces histogram detectability artifacts of naive LSB substitution.
- [34] MinIO Inc. MinIO: High-performance object storage. <https://min.io/>, 2019. S3-compatible open-source object storage; used as Stegnar’s carved image payload storage backend.
- [35] Steven J. Murdoch and Stephen Lewis. Embedding covert channels into TCP/IP. In *Proceedings of Information Hiding: 7th International Workshop (IH)*, pages 247–261. Springer, 2005. Demonstrates IP Identification field as a practical storage covert channel; foundational network steganography paper.
- [36] Tomas Pevný, Tomáš Filler, and Patrick Bas. Using high-dimensional image models to perform highly undetectable steganography. In *Proceedings of Information Hiding: 12th International Conference (IH)*, pages 161–177, 2010. HUGO (Highly Undetectable steGO): introduces additive distortion minimization paradigm; predecessor to WOW and UNIWARD schemes.
- [37] Ponemon Institute. 2022 cost of insider threats: Global report. Technical report, Ponemon Institute LLC, 2022. Insider threat cost and dwell time analysis; supports attacker profile definition in Stegnar threat model. Reports average insider threat dwell time exceeding 85 days.
- [38] Yinlong Qian, Jing Dong, Wei Wang, and Tieniu Tan. Deep learning for steganalysis via convolutional neural networks. In *Proceedings of SPIE: Media Watermarking, Security, and Forensics*, volume 9409, page 94090J, 2015. Early application of CNNs to steganalysis; demonstrates viability of learned feature approach over hand-crafted features.
-

-
- [39] Craig H. Rowland. Covert channels in the TCP/IP protocol suite. *First Monday*, 2(5), 1997. TCP ISN and IP TTL field covert channels; early documentation of protocol-level steganography techniques.
- [40] Krzysztof Szczypiorski. HICCUPS: Hidden communication system for corrupted networks. In *Proceedings of the 10th International Multi-Conference on Advanced Computer Systems (ACS)*, pages 31–40, 2003. HICCUPS: multi-protocol network steganography system using corrupted/reserved frame fields.
- [41] Weixuan Tang, Bin Li, Shunquan Tan, Mauro Barni, and Jiwu Huang. CNN-Based adversarial embedding for image steganography. *IEEE Transactions on Information Forensics and Security*, 14(8):2074–2087, 2019. Adversarial steganography: embedding optimized against CNN-based steganalysts; defines the upper boundary of Stegnar’s sophistication assumption.
- [42] Marcos A. M. Vieira, Matheus S. Castanho, Racyus D. G. Pacífico, Elerson R. S. Santos, Eduardo P. M. Câmara Júnior, and Luiz F. M. Vieira. Fast packet processing with eBPF and XDP: Concepts, code, challenges, and applications. *ACM Computing Surveys*, 53(1):1–36, 2020. Comprehensive survey of eBPF/XDP for kernel-level packet processing; technical foundation for Stegnar MITM gateway’s traffic redirection mechanism.
- [43] Yong Wang, Wen Liu, Ru Zhu, and Yi Yu. Detecting covert timing channels in network traffic. In *Proceedings of the International Conference on Computational Intelligence and Security (CIS)*, pages 173–177. IEEE, 2010. Flow-level behavioral analysis for covert timing channel detection; highlights limitations of traffic-behavior-only approaches without payload inspection.
- [44] Andreas Westfeld and Andreas Pfitzmann. Attacks on steganographic systems. In *Proceedings of Information Hiding: 3rd International Workshop (IH)*, pages 61–76. Springer, 1999. Chi-square attack: first practical statistical steganalysis; detects naive sequential LSB embedding via histogram equalization artifacts.
- [45] Guanshuo Xu, Han-Zhou Wu, and Yun-Qing Shi. Structural design of convolutional neural networks for steganalysis. volume 23, pages 708–712. IEEE, 2016. XuNet: first competitive CNN architecture for spatial-domain steganalysis, supersedes rich model approaches for several schemes.
- [46] Jian Ye, Jiangqun Ni, and Ming Yang. Deep learning hierarchical representations for image steganalysis. volume 12, pages 2545–2557. IEEE, 2017. Ye-Net: introduces truncated linear activation function for steganalysis CNNs; strong performance on spatial-domain schemes.
-

-
- [47] Zhili Zhang, Guanshuo Xu, Yanli Chen, and Yun-Qing Shi. Udh: Universal deep hiding for steganography, watermarking, and light field messaging. In *Proceedings of Advances in Neural Information Processing Systems (NeurIPS)*, pages 10223–10234, 2020. Neural network-based steganographic hiding; represents emerging class of deep learning-powered steganography that challenges traditional CNN steganalysis.
-